



Natural Language Processing with Recurrent Neural Networks

2019–20

Ion Androutsopoulos

<http://www.aueb.gr/users/ion/>

Contents

- Recurrent neural networks (RNNs), GRUs/LSTMs.
- Applications in POS tagging and named entity recognition.
- RNN language models.
- RNNs with self-attention and applications in text classification.
- Bidirectional and stacked RNNs.
- Obtaining word embeddings from character-based RNNs.
- Variational dropout.
- Hierarchical RNNs for text classification and sequence labeling.
- Sequence-to-sequence RNN models with attention, and applications in machine translation.
- Universal sentence encoders, LASER.
- Pretraining language models, context-aware embeddings, ELMo.

Extracting contract elements

THIS AGREEMENT is made the 15th day of October 2009
(The “Effective Date”) BETWEEN:

- (1) Sugar 13 Inc., a corporation whose office is at James House, 42-50 Bond Street, London, EW2H TL (“Sugar”);
- (2) E2 UK Limited, a limited company whose registered office is at 260 Bathurst Road, Yorkshire, SL3 4SA (“Provider”).

RECITALS:

- A. The Parties wish to enter into a framework agreement which will enable Sugar, from time to time, to [...]
- B. [...]

NO THEREFORE IT IS AGREED AS FOLLOWS:

ARTICLE I - DEFINITIONS

- “Sugar” shall mean: Sugar 13 Inc.
- “Provider” shall mean: E2 UK Limited
- “1933 Act” shall mean: Securities Act of 1933

ARTICLE II - TERMINATION

The Service Period will be for five (5) years from the Effective Date (The “Initial Term”). The agreement is considered to be terminated in October 16, 2014.

ARTICLE III - PAYMENT - FEES

During the service period monthly payments should occur. The estimated fees for the Initial Term are £154,800.

ARTICLE IV - GOVERNING LAW

This agreement shall be governed and construed in accordance with the Laws of England & Wales. Each party hereby irrevocably submits to the exclusive jurisdiction of the courts sitting in Northern London.

IN WITNESS WHEREOF, the parties have caused their respective duly authorized officers to execute this Agreement.

BY: George Fake
Authorized Officer
Sugar 13 Inc.

BY: Olivier Giroux
CEO
E2 UK LIMITED

Identify start/end dates,
duration, contractors, amount,
legislations refs, jurisdiction
etc. Similar to Named Entity
Recognition (NER).

Window-based NER example

i -th word of the text being classified

3-word **window** (often larger)

yesterday language **tech** announced that...

1-hot vectors ($|V| \times 1$) of the words in the **window**. ($|V|$ is the **vocabulary size**).

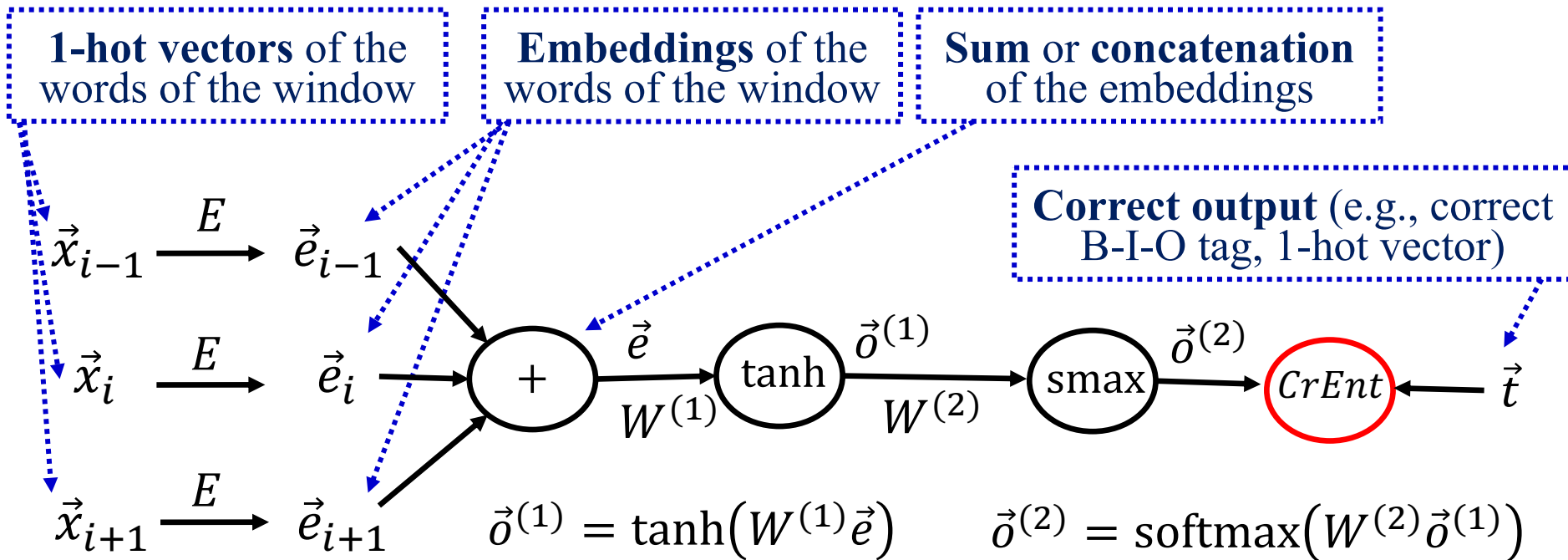
$$\vec{x}_{i-1} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \dots \\ 0 \end{bmatrix} \quad \vec{x}_i = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \dots \\ 0 \end{bmatrix} \quad \vec{x}_{i+1} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \dots \\ 1 \end{bmatrix}$$

Embeddings ($d \times 1$) of the words in the **window**. (d is the **dimensionality** of the word embeddings).

$$\vec{e}_{i-1} = \begin{bmatrix} 1.8 \\ 2.3 \\ -1.4 \\ 3.7 \\ \dots \\ -1.1 \end{bmatrix} \quad \vec{e}_i = \begin{bmatrix} 2.4 \\ -3 \\ 9.3 \\ 5.1 \\ \dots \\ 3.9 \end{bmatrix} \quad \vec{e}_{i+1} = \begin{bmatrix} 2.2 \\ 3.8 \\ 1.2 \\ -6.4 \\ \dots \\ 7.1 \end{bmatrix}$$

Let E be a matrix ($d \times |V|$) that **contains** all the **embeddings** of the **vocabulary** as **columns**. Then:
 $\vec{e}_{i-1} = E\vec{x}_{i-1}$, $\vec{e}_i = E\vec{x}_i$, ...

Window-based NER example



We learn $W^{(1)}$, $W^{(2)}$ with **backpropagation**. We can also learn (or modify) the **word embeddings** E during **backpropagation**! But when we don't have large training datasets (e.g., corpus manually annotated with B-I-O tags), it may be better to use **pre-trained embeddings**, which can be obtained from large non-annotated corpora (e.g., via Word2Vec, GloVe, to be discussed).

We can use the same window-based approach for **POS-tagging**, **chunking**, ...

Cross-entropy loss

Word being classified.

3-word window (often larger).

yesterday language **tech** announced that...

$$\vec{o} = \begin{bmatrix} P_m(C = c_1) \\ P_m(C = c_2) \\ P_m(C = c_3) \\ \dots \\ P_m(C = c_k) \end{bmatrix} = \begin{bmatrix} 0.05 \\ 0.12 \\ 0.08 \\ \dots \\ 0.14 \end{bmatrix}$$

Probability estimates produced by the **classifier** for the class of the word “tech”.

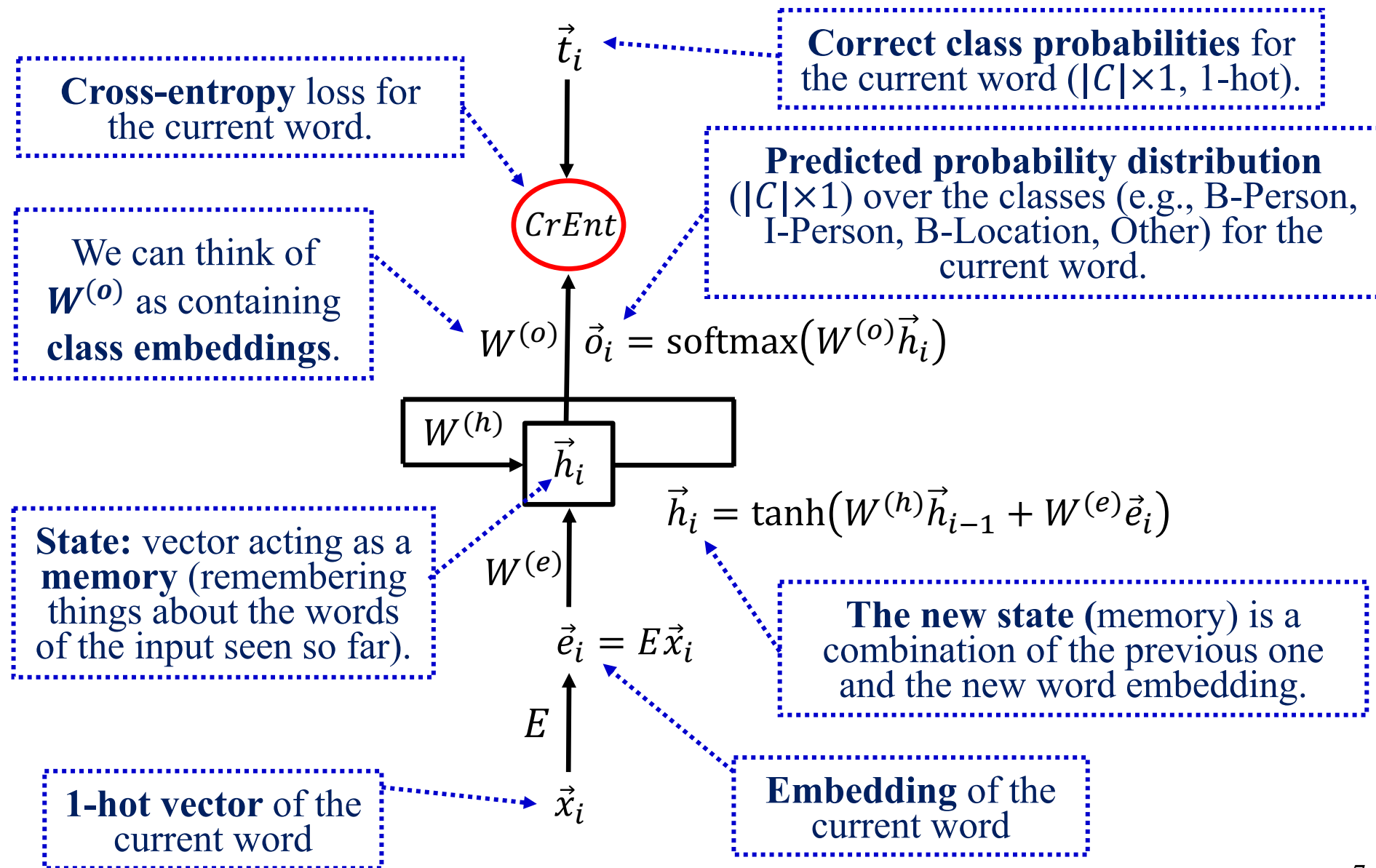
The **correct “probabilities”** for the class of “tech”. A **1-hot vector**.

$$\vec{t} = \begin{bmatrix} P(C = c_1) \\ P(C = c_2) \\ P(C = c_3) \\ \dots \\ P(C = c_k) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \dots \\ 0 \end{bmatrix}$$

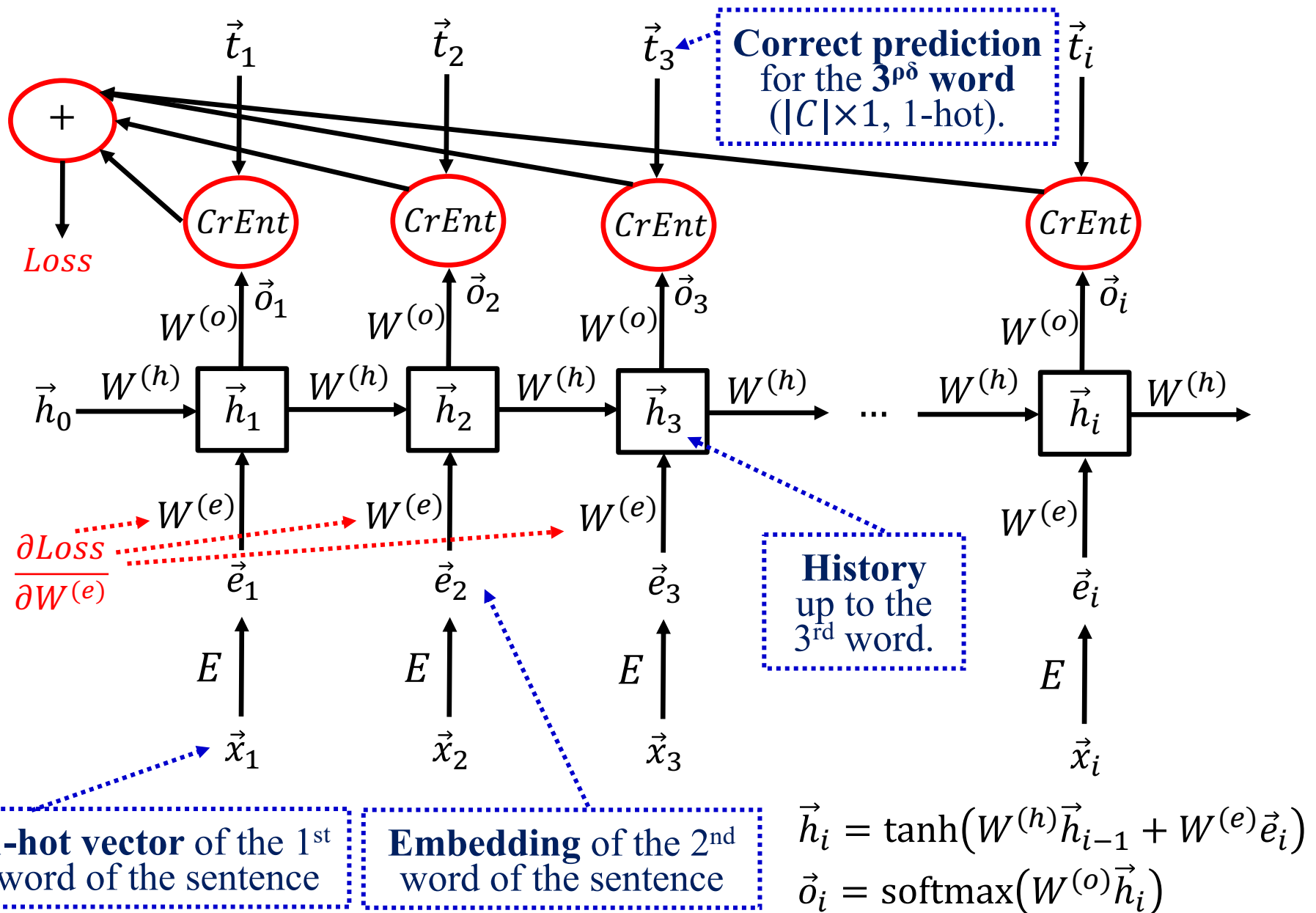
The **log-likelihood** of the **correct class** according to the **classifier** (with a minus sign).

$$H_{P_m}(C) = - \sum_{i=1}^k P(C = c_i) \log_2 P_m(C = c_i) = - \log_2 P_m(C = c_2)$$

RNN-based Named Entity Recognizer



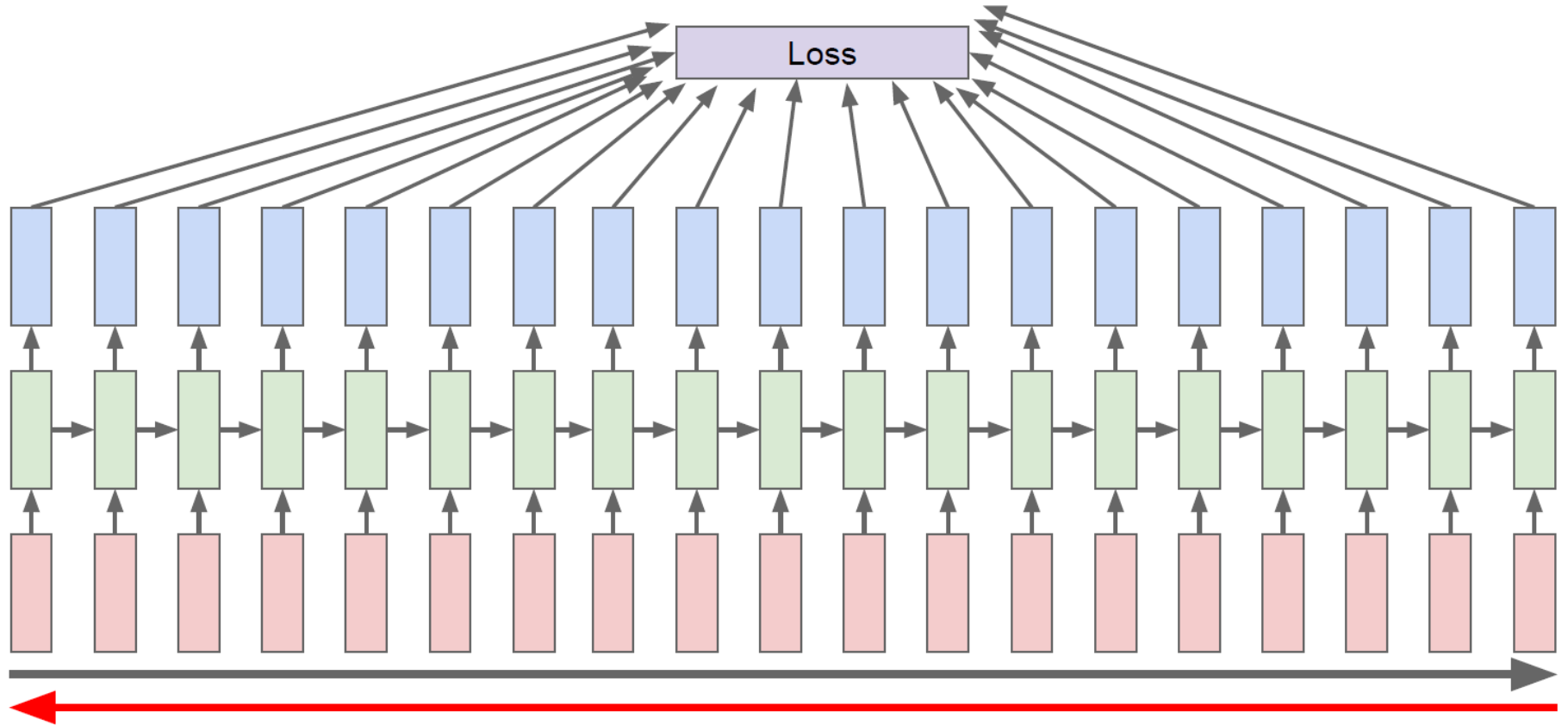
Unrolled RNN



Unrolled RNN

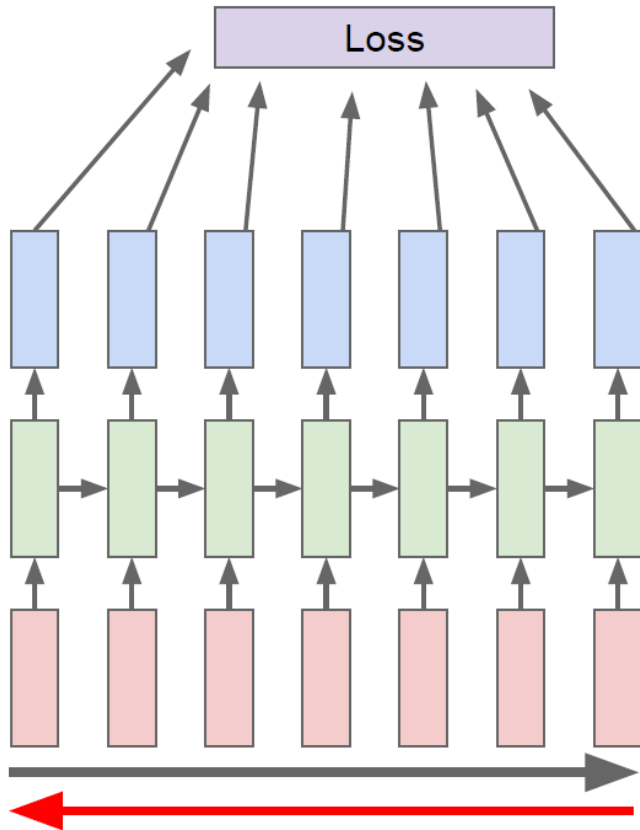
Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Unrolled RNN in practice

Truncated Backpropagation through time

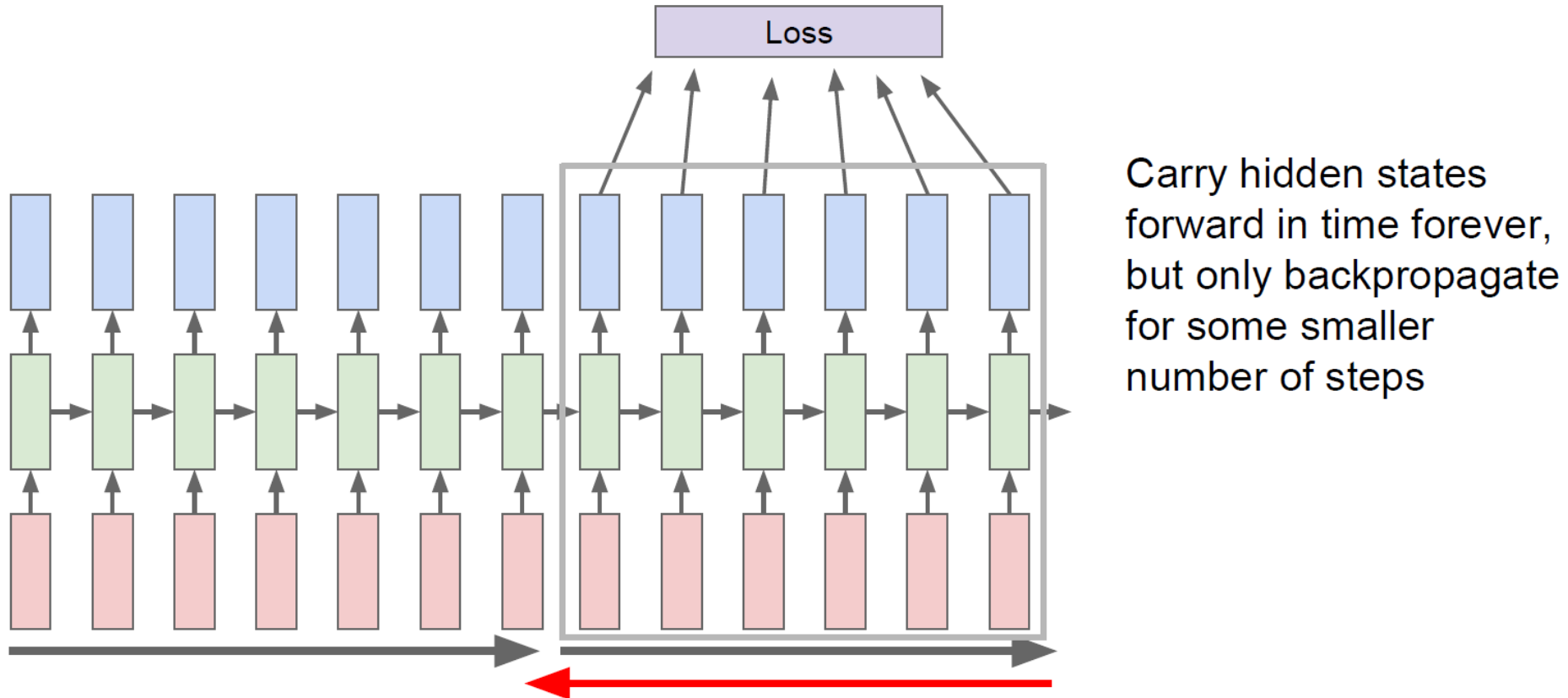


Run forward and backward through chunks of the sequence instead of whole sequence

ion: Most NN platforms require all **sequences** (e.g., sentences) to have the **same length** (fixed number of time-steps) or up to a **maximum length**. For **smaller** sequences, e.g., **pad** with dummy tokens (and ignore the pads in the loss). For **larger** sequences, we can do **truncated back-prop**, with chunks that fit the available memory (also faster).

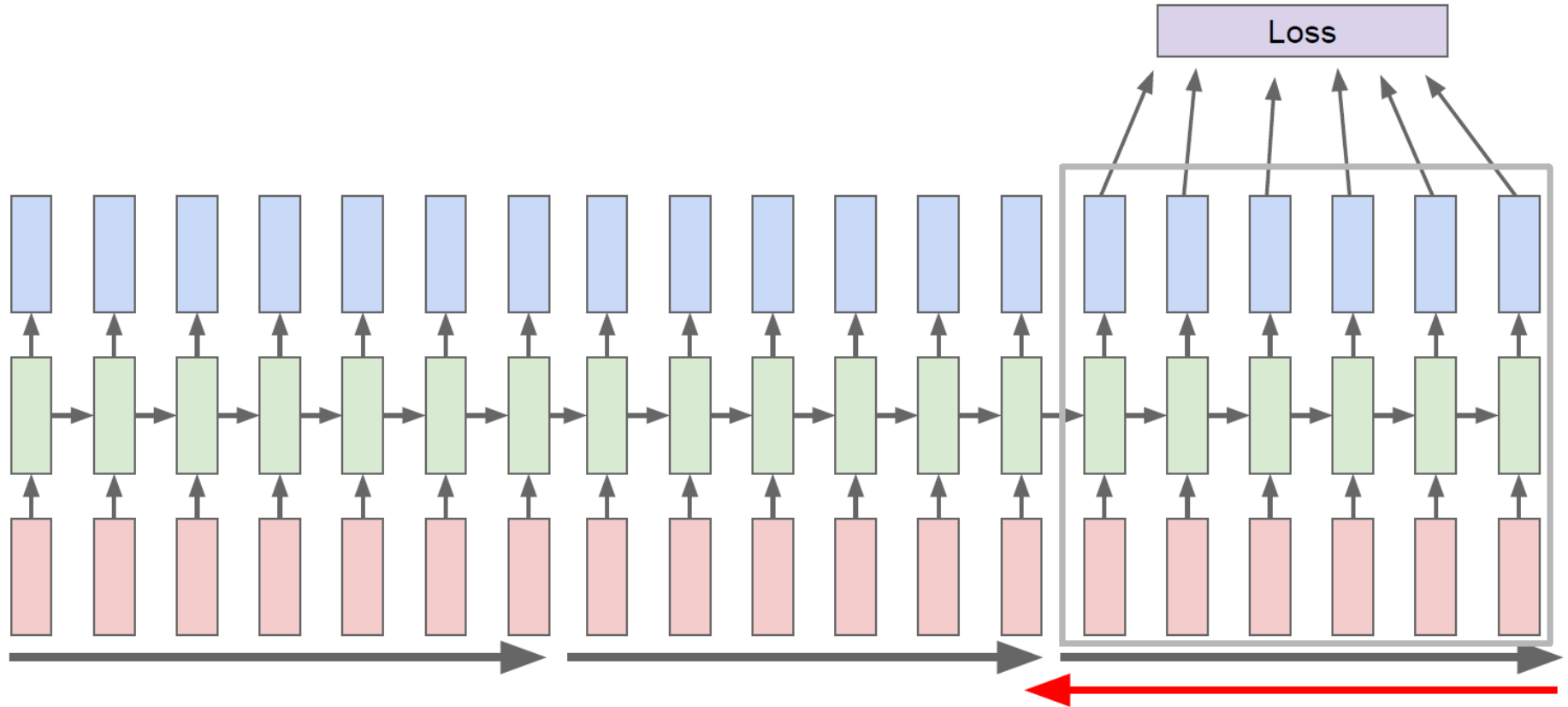
Unrolled RNN in practice

Truncated Backpropagation through time

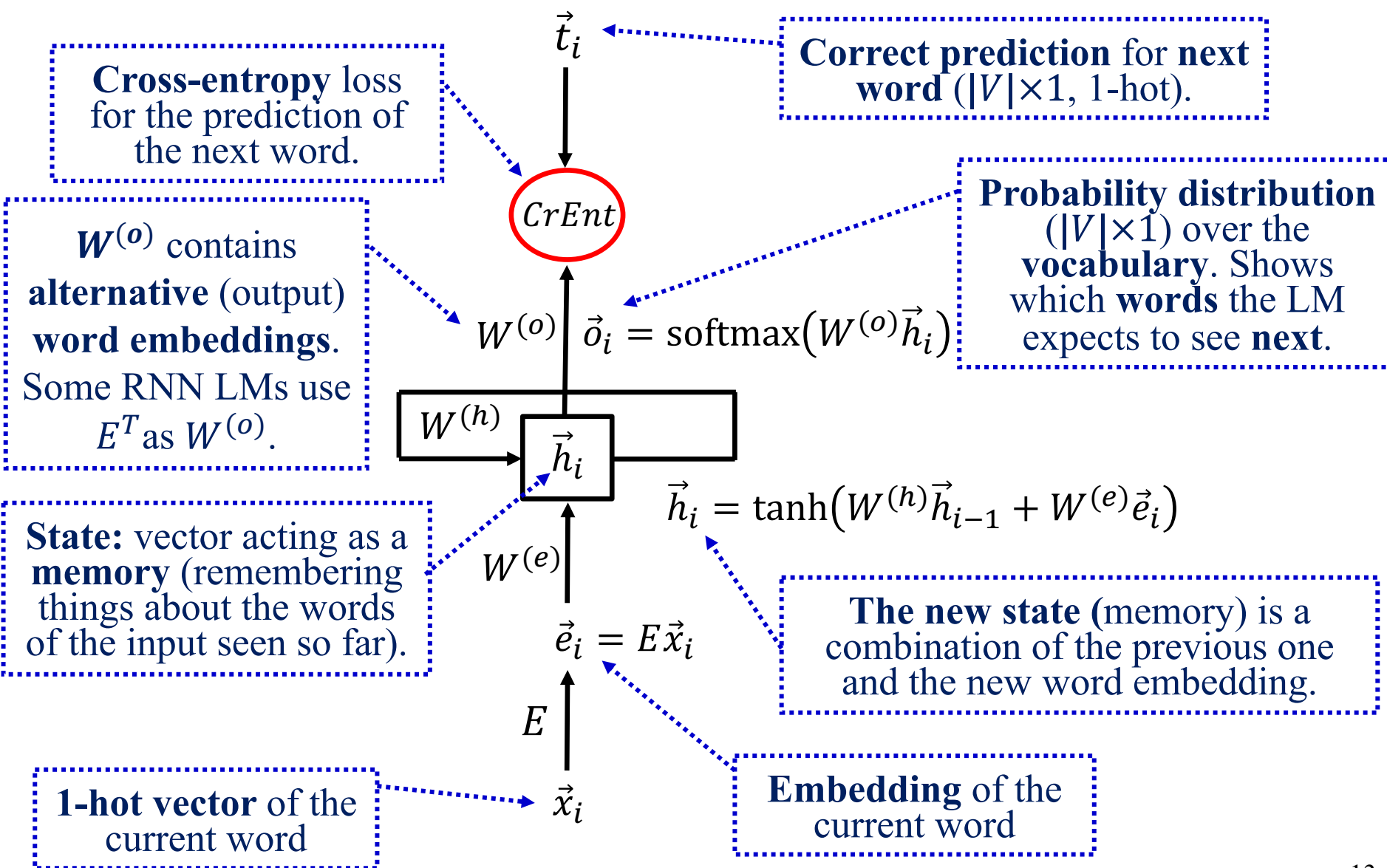


Unrolled RNN in practice

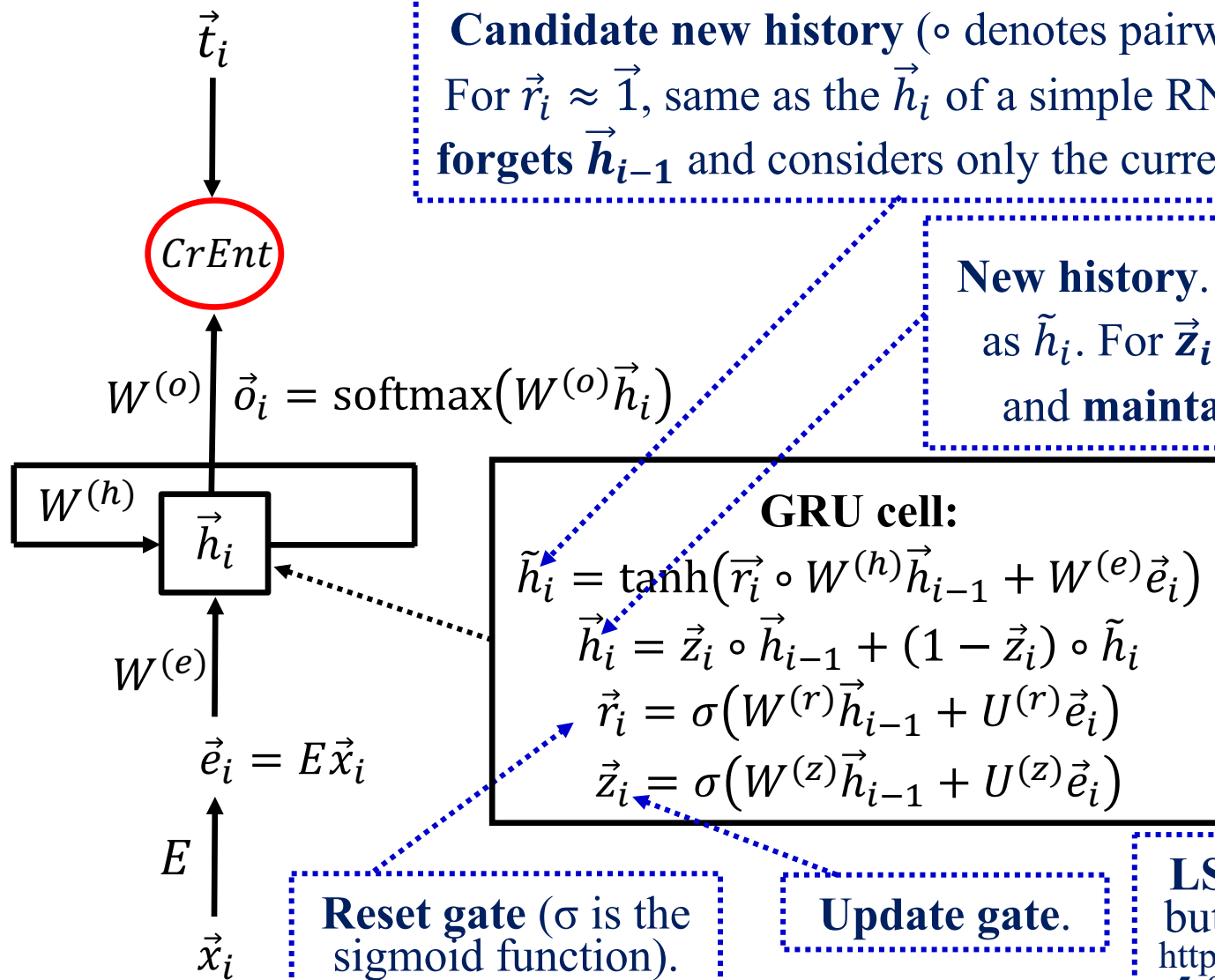
Truncated Backpropagation through time



RNN language model



RNN LM with GRU cells



Candidate new history ($\circ$ denotes pairwise multiplication). For $\vec{r}_i \approx \vec{1}$, same as the $\vec{h}_i$ of a simple RNN cell. For $\vec{r}_i \approx \vec{0}$, **forgets** $\vec{h}_{i-1}$ and considers only the current word embedding.

New history. For $\vec{z}_i \approx \vec{0}$, same as $\tilde{h}_i$. For $\vec{z}_i \approx \vec{1}$, ignores $\tilde{h}_i$ and **maintains** $\vec{h}_{i-1}$ as $\vec{h}_i$.

Reset gate (σ is the sigmoid function).

Update gate.

LSTM cells are similar, but with **more gates**. See <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

More about RNNs

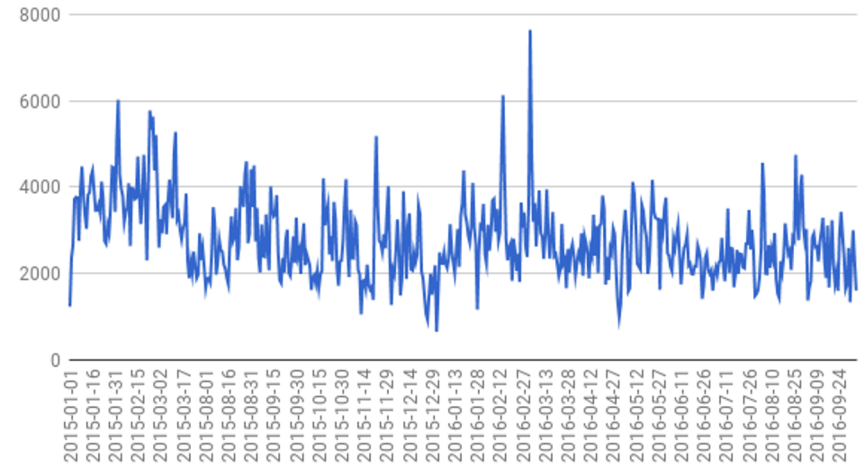
- Trained by **backpropagation** (with **unrolled** view).
 - For **each sentence (or window)**, feed it to the **unrolled RNN**, compute the **loss** and **backpropagate**, adding gradients obtained for the **same matrix** (e.g., same $W^{(h)}$ at each cell).
 - **GRU** or **LSTM** cells help avoid **vanishing gradients**.
 - The norms of the **gradients** can be **clipped** (when larger than a max value) to avoid **exploding gradients**.
- We can also **learn the word embeddings (E)** with an RNN LM. Billions of **free training examples**!
 - We can then use the **(pre-trained) embeddings** in **other NLP applications**, keeping the embeddings frozen or updating them (but this may move some words, e.g., not their synonyms).
 - With a **large vocabulary**, **softmax** is too **slow** (alternatives: small vocabulary, hierarchical softmax, negative sampling).

User comment moderation

A moderation panel assists the moderators to detect abusive comments, and leads to quicker publication of non-abusive comments.

Highlighting suspicious words using an RNN with self-attention.

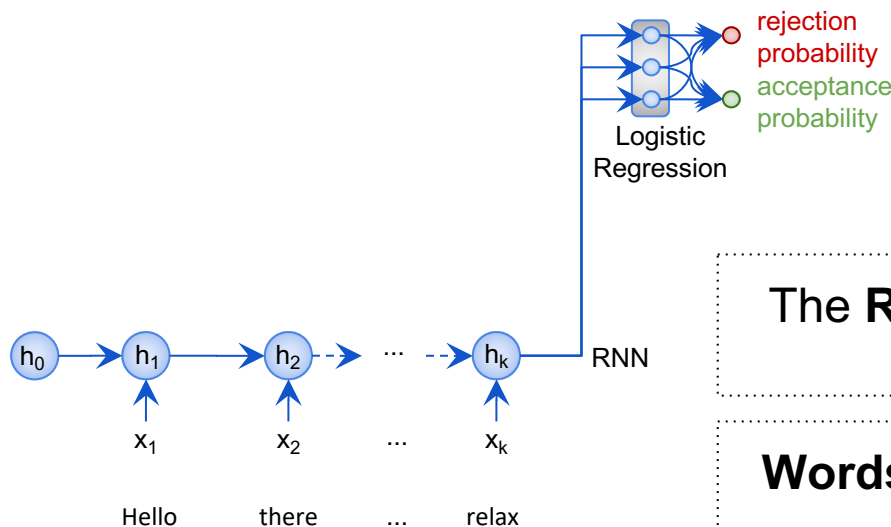
Number of comments per day



Moderation Panel

Go	and	hang	yourself	!						85%		
You	are	ignorant	and	vandal	!	Stop	it	!		88%		
Hello	there	try	to	relax						0%		
Thanks	.	Please	go	f#\$@	yourself	.	Ty	!		85%		

RNN-based moderation of user comments



A **Logistic Regression (LR)** layer uses the **last state** of the RNN as a feature vector.

The **RNN states** “summarize” the **words seen**. We use **GRUs** (128 dims).

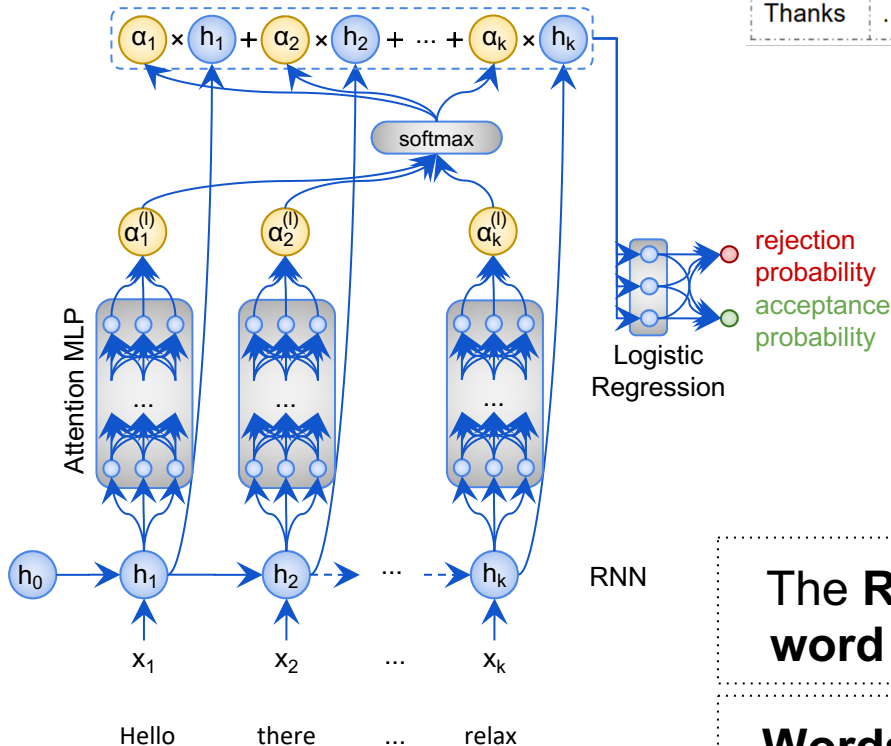
Words are mapped to **embeddings** (word2vec, 300 dimensions).

J. Pavlopoulos, P. Malakasiotis and I. Androutsopoulos,, “Deeper Attention to Abusive User Content Moderation”, EMNLP 2017, <http://nlp.cs.aueb.gr/pubs/emnlp2017.pdf>.

RNN with deep self-attention (α -RNN)

The **attention scores** can also be used to **highlight suspicious words!**

Go	and	hang	yourself	!				
You	are	ignorant	and	vandal	!	Stop	it	!
Thanks	.	Please	go	fuck	yourself	.	ty	!



An **LR layer** uses the **attention-weighted sum** of the **RNN states** as a feature vector.

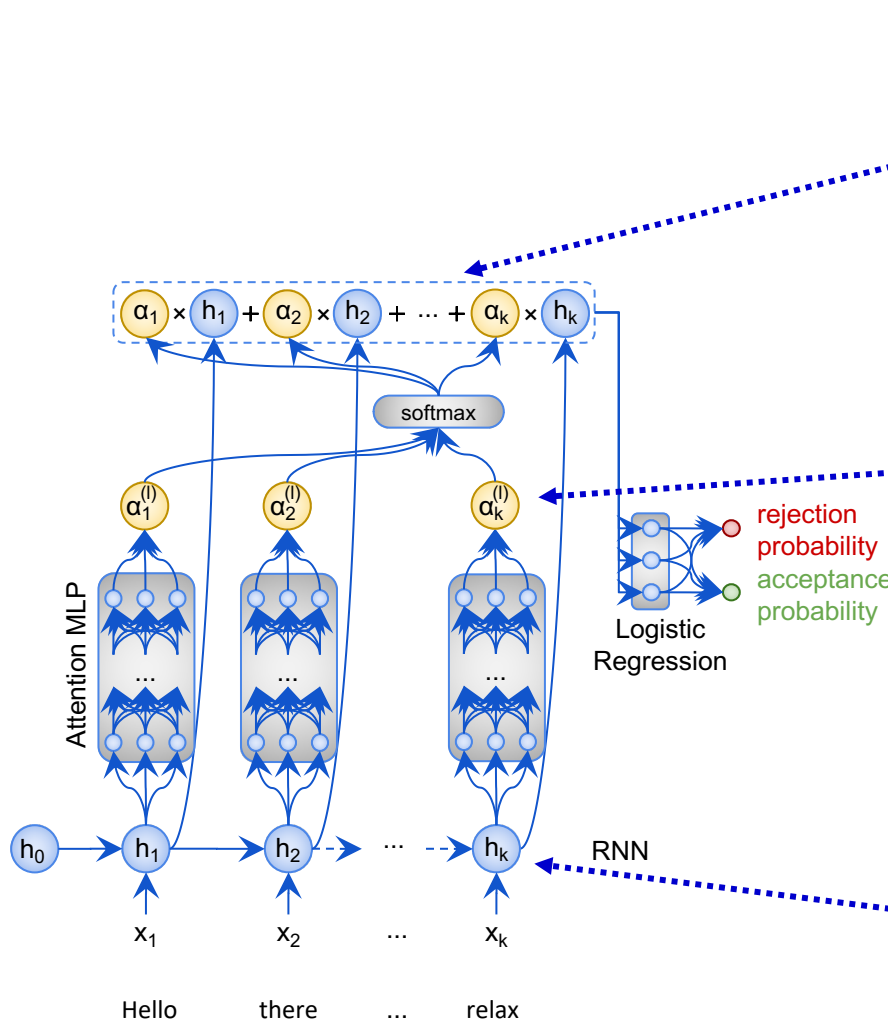
Each state of the RNN is used as **input** to a **self-attention MLP** (4 layers, same **MLP** at each **time step**) that scores the **importance** of the corresponding **word**.

The **RNN states** act as (left-)context sensitive **word embeddings**. We use **GRUs** (128 dims).

Words are mapped to **embeddings** (word2vec, 300 dimensions).

J. Pavlopoulos, P. Malakasiotis and I. Androutsopoulos,, “Deeper Attention to Abusive User Content Moderation”, EMNLP 2017, <http://nlp.cs.aueb.gr/pubs/emnlp2017.pdf>.

RNN with deep self-attention (α -RNN)



$$h_{sum} = \sum_{t=1}^k a_t h_t$$

$$P_{a-RNN}(reject|c) = \sigma(W_p h_{sum} + b_p)$$

$$a_t^{(1)} = \text{RELU}(W^{(1)} h_t + b^{(1)})$$

$$\dots$$

$$a_t^{(l-1)} = \text{RELU}(W^{(l-1)} a_t^{(l-2)} + b^{(l-1)})$$

$$a_t^{(l)} = W^{(l)} a_t^{(l-1)} + b^{(l)}$$

$$a_t = \text{softmax}(a_t^{(l)}; a_1^{(l)}, \dots, a_k^{(l)})$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

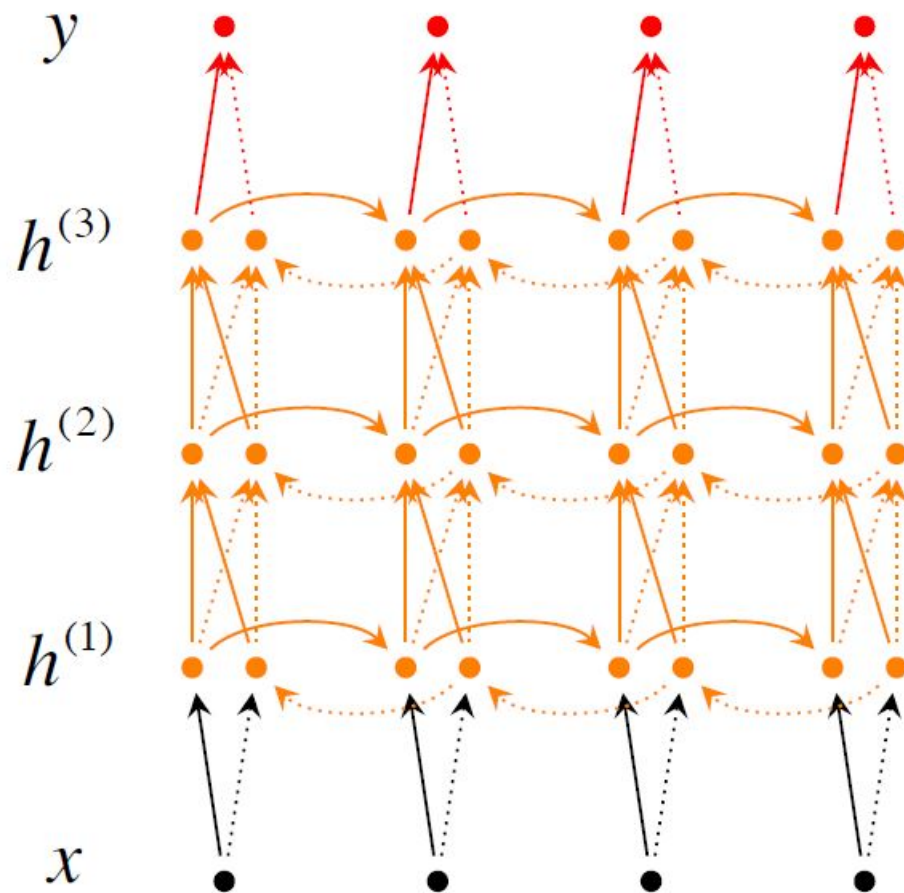
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

J. Pavlopoulos, P. Malakasiotis and I. Androutsopoulos,, “Deeper Attention to Abusive User Content Moderation”, EMNLP 2017, <http://nlp.cs.aueb.gr/pubs/emnlp2017.pdf>.

Deep Bidirectional RNNs by Irsoy and Cardie



From the slides of R. Socher's course "Deep Learning for NLP", 2015. <http://cs224d.stanford.edu/>

$$\vec{h}_t^{(i)} = f(\vec{W}^{(i)} h_t^{(i-1)} + \vec{V}^{(i)} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)})$$

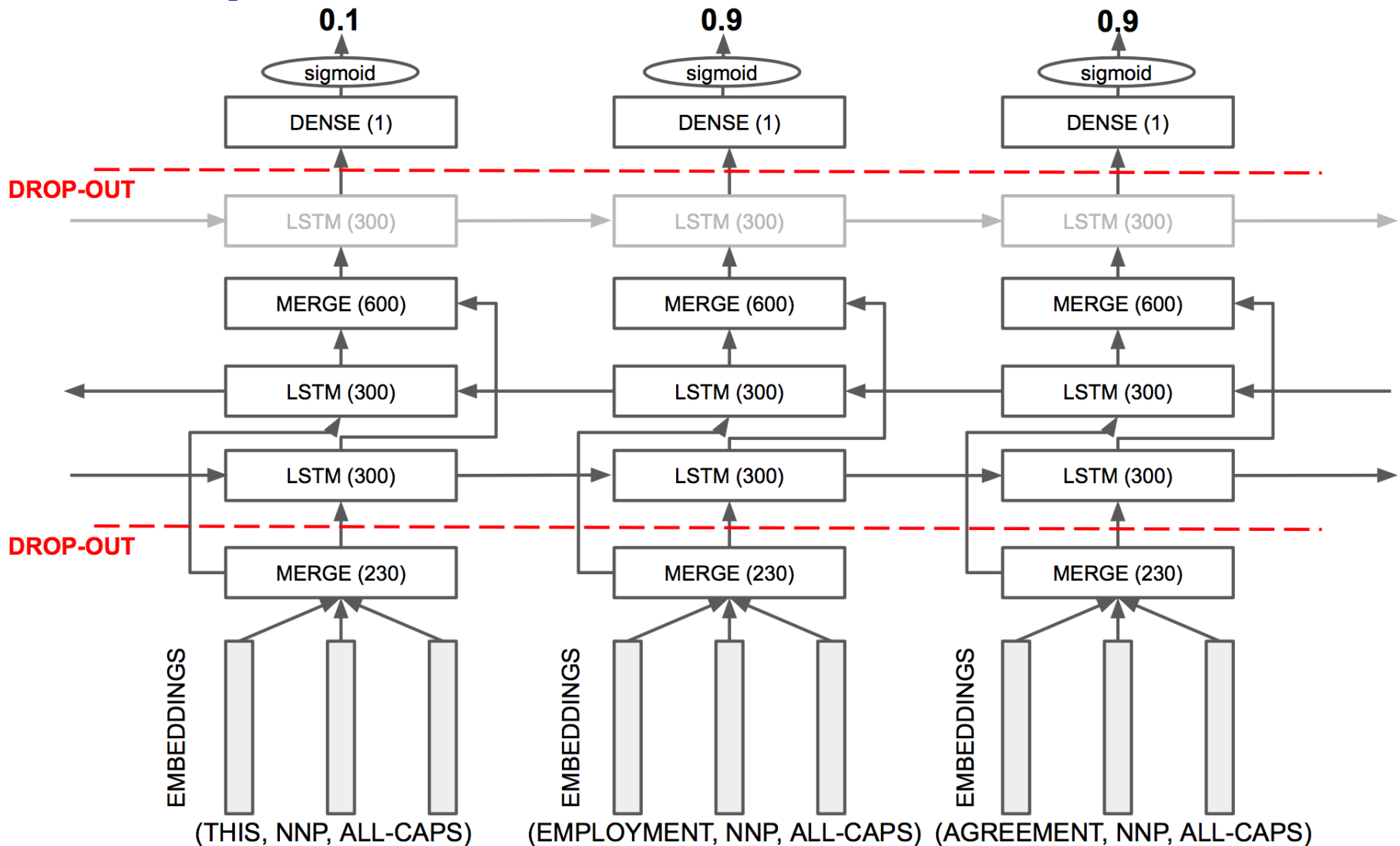
$$\overleftarrow{h}_t^{(i)} = f(\overleftarrow{W}^{(i)} h_t^{(i-1)} + \overleftarrow{V}^{(i)} \overleftarrow{h}_{t+1}^{(i)} + \overleftarrow{b}^{(i)})$$

$$y_t = g(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c)$$

E.g., probabilities of B, I, O tags at t -th word of the sequence.

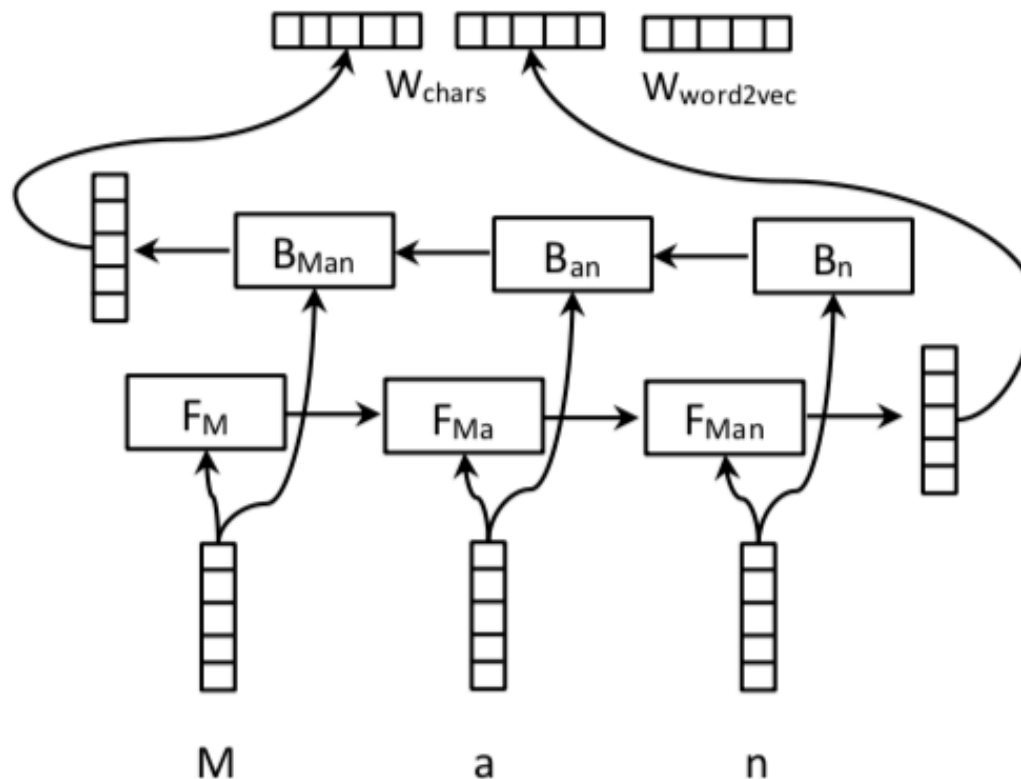
Each memory layer passes an intermediate sequential representation to the next.

Improved contract element extraction



I. Chalkidis and I. Androutsopoulos, "A Deep Learning Approach to Contract Element Extraction", JURIX 2017, <http://nlp.cs.aueb.gr/pubs/jurix2017.pdf>.

RNNs that produce word embeddings from character embeddings



Word embedding layer,
part of a larger network.

We **concatenate** the
word embedding we get
from the **character-level**
biLSTM with the
Word2Vec embedding.

The **character**
embeddings are **learned**
during back-propagation.

G, Bekoulis, J, Deleu, T, Demeester, C. Develder, “Joint entity recognition and relation extraction as a multi-head selection problem”, Expert Systems with Applications, Vol, 114, pp. 34-45, 2018. Figure from the pre-print <https://arxiv.org/abs/1804.07847>.

Dropout vs. Variational Dropout

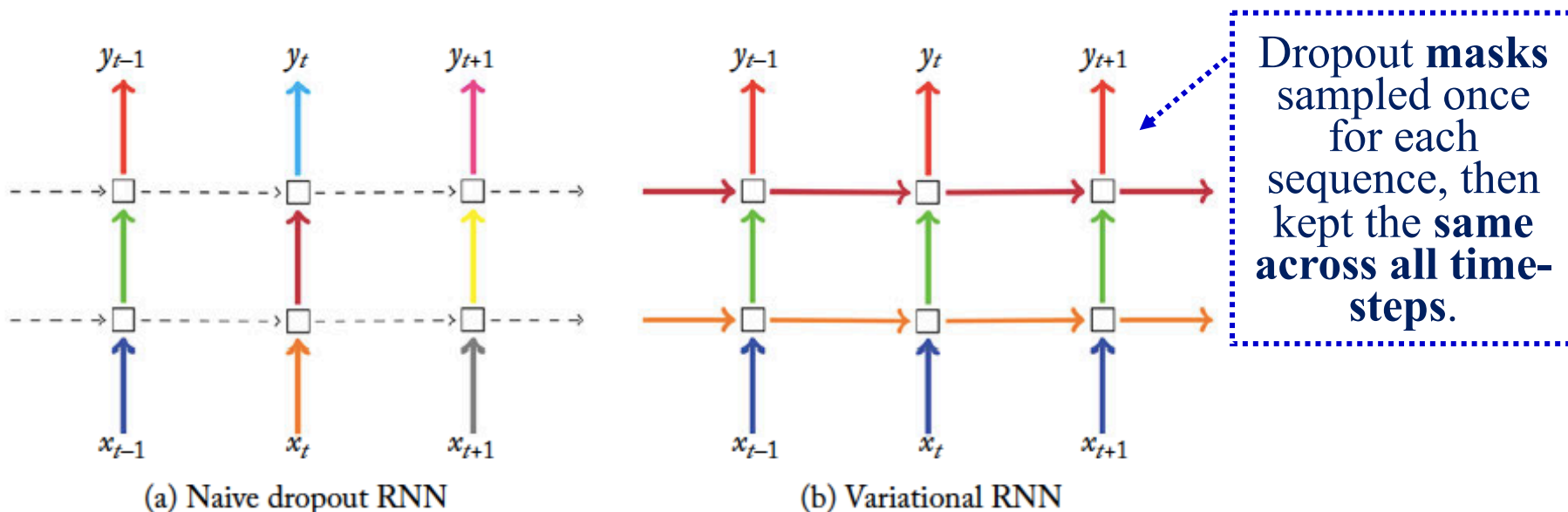
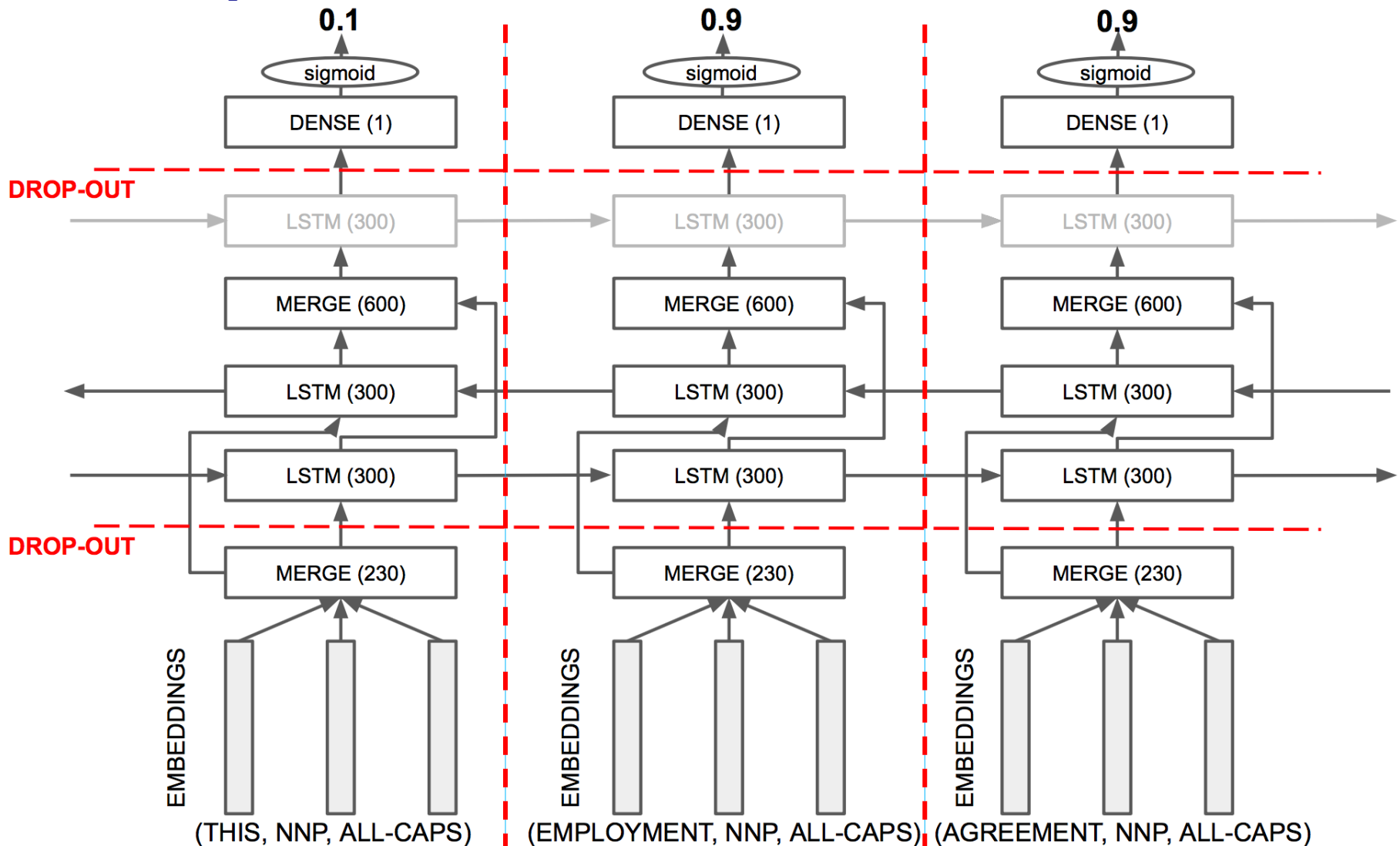


Figure 15.2: Gal's proposal for RNN dropout (b), vs. the previous suggestion by Pham et al. [2013], Zaremba et al. [2014] (a). Figure from Gal [2015], used with permission. Each square represents an RNN unit, with horizontal arrows representing time dependence (recurrent connections). Vertical arrows represent the input and output to each RNN unit. Colored connections represent dropped-out inputs, with different colors corresponding to different dropout masks. Dashed lines correspond to standard connections with no dropout. Previous techniques (naive dropout, left) use different masks at different time steps, with no dropout on the recurrent layers. Gal's proposed technique (Variational RNN, right) uses the same dropout mask at each time step, including the recurrent layers.

From: Y. Goldberg, *Neural Network Models for Natural Language Processing*, Morgan & Claypool Publishers, 2017.

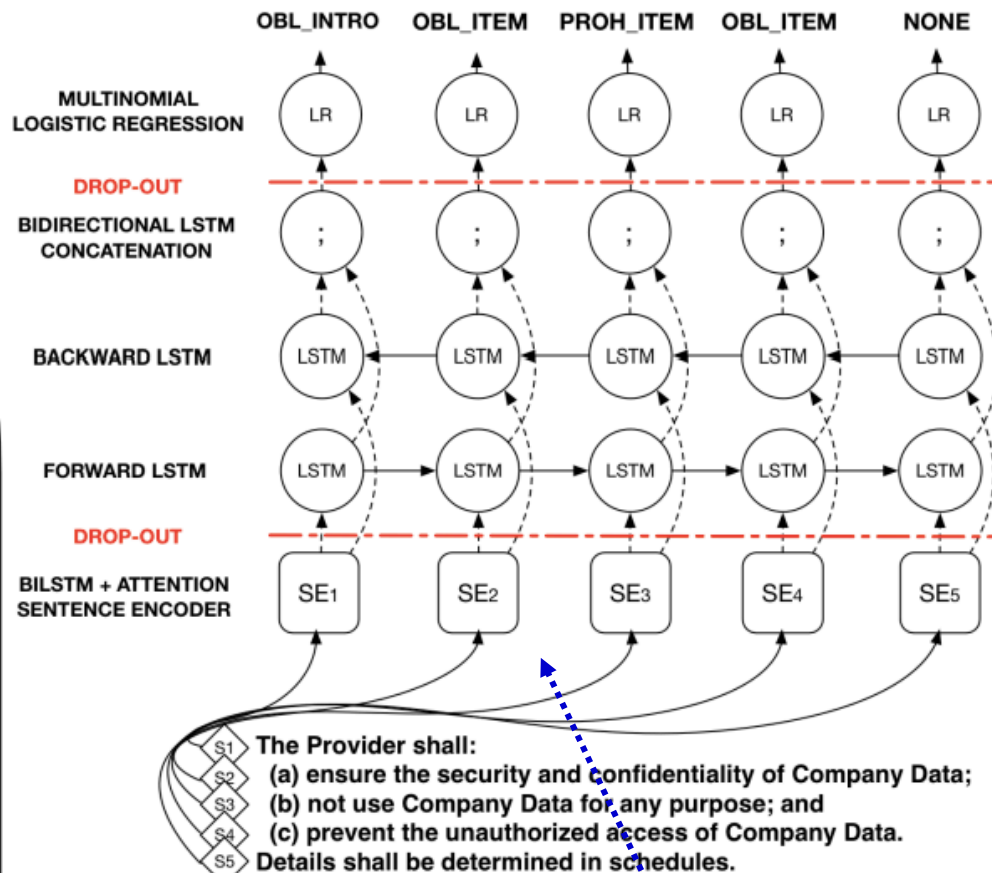
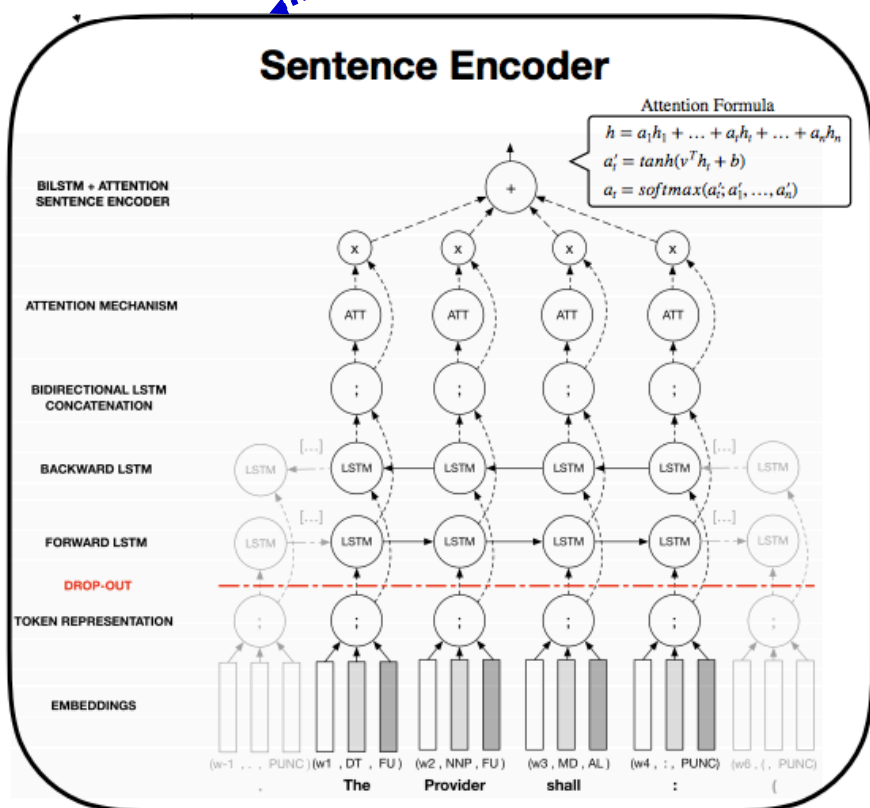
Improved contract element extraction



I. Chalkidis and I. Androutsopoulos, “A Deep Learning Approach to Contract Element Extraction”, JURIX 2017, <http://nlp.cs.aueb.gr/pubs/jurix2017.pdf>.

Sequence labeling with a Hierarchical RNN

The **lower RNN** reads the words of each sentence and converts the sentence to a **sentence embedding**.



The **upper RNN** reads a sequence of sentence embeddings and **classifies each sentence**.

I. Chalkidis, I. Androutsopoulos, A. Michos, "Obligation and Prohibition Extraction Using Hierarchical RNNs", ACL 2018. <http://www.aclweb.org/anthology/P18-2041>

Sequence labeling with a Hierarchical RNN

Case ID: 001-148227 **Violated Articles:** Article 3 **Predicted Violation:** YES (0.97%)

-
1. The applicant was born in 1955 and lives in **Kharkiv**.
2. On 5 May 2004 the applicant was arrested by four police officers on **suspicion** of bribe - taking .
The police officers took him to the **Kharkiv Dzerzhynskyy District Police Station** , where he was held **overnight** .
According to the applicant , the police officers beat him for several hours , forcing him to confess .
3. On 6 May 2004 the applicant was taken to the **Kharkiv City Prosecutor's Office** . He complained of ill-treatment to a senior prosecutor from the above office . The prosecutor referred the **applicant** for a forensic medical examination .
4. On 7 May 2004 the applicant was diagnosed with **concussion** and admitted to **hospital** .
5. On 8 May 2004 the applicant underwent a forensic medical examination , which established that he had numerous **bruises** on his face , chest , legs and arms , as well as a **damaged** tooth .
6. On 11 May 2004 criminal proceedings were instituted against the applicant on charges of bribe-taking . They were eventually terminated on 27 April 2007 for lack of **corpus delicti** .
7. On 2 June 2004 the applicant lodged another complaint of ill - treatment with the **Kharkiv City Prosecutor's Office** .

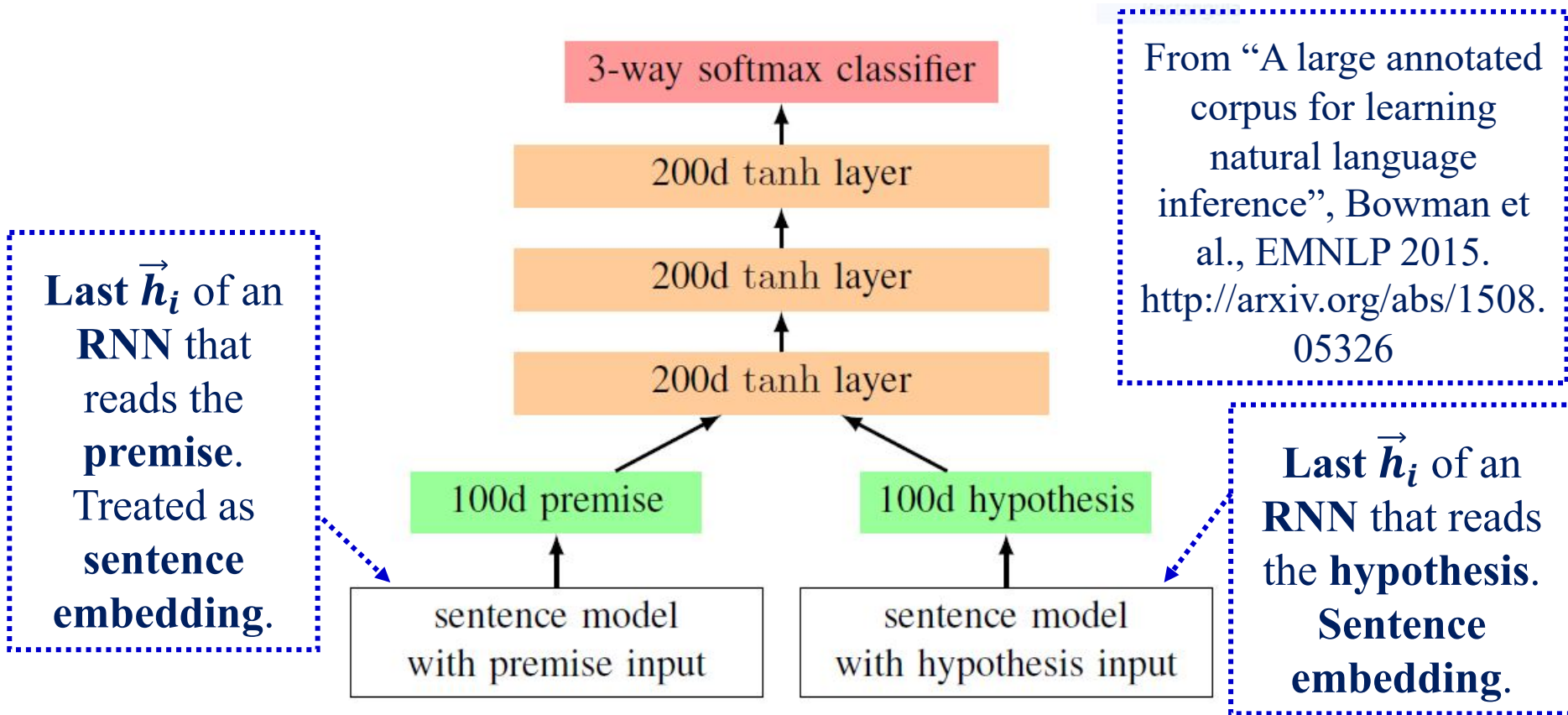
Figure 1: Attention over words (colored words) and facts (vertical heat bars) as produced by HAN.

Words with high attention scores.

Sentences with high attention scores.

I. Chalkidis, I. Androustopoulos and N. Aletras, "Neural Legal Judgment Prediction in English", ACL 2019. <https://www.aclweb.org/anthology/P19-1424/>

Textual entailment with RNNs/MLP



A man inspects the uniform of a figure in some East Asian country.

contradiction
C C C C C

The man is sleeping

An older and younger man smiling.

neutral
N N E N N

Two men are smiling and laughing at the cats playing on the floor.

A black race car starts up in front of a crowd of people.

contradiction
C C C C C

A man is driving down a lonely road.

A soccer game with multiple males playing.

entailment
E E E E E

Some men are playing a sport.

A smiling costumed woman is holding an umbrella.

neutral
N N E C N

A happy woman in a fairy costume holds an umbrella.

RNNs for Machine Translation

From the slides of R. Socher's course "Deep Learning for NLP", 2015. <http://cs224d.stanford.edu/>

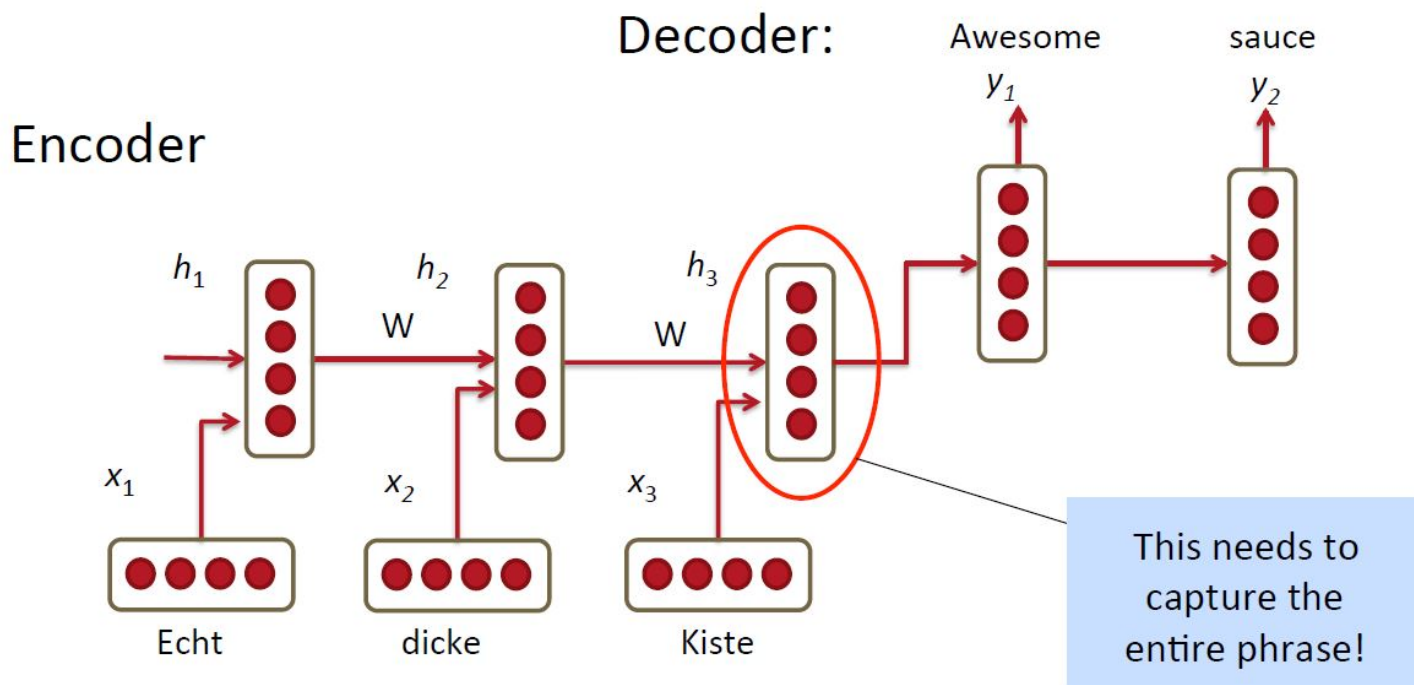
Encoder: $h_t = \phi(h_{t-1}, x_t) = f(W^{(hh)}h_{t-1} + W^{(hx)}x_t)$

Decoder: $h_t = \phi(h_{t-1}) = f(W^{(hh)}h_{t-1})$

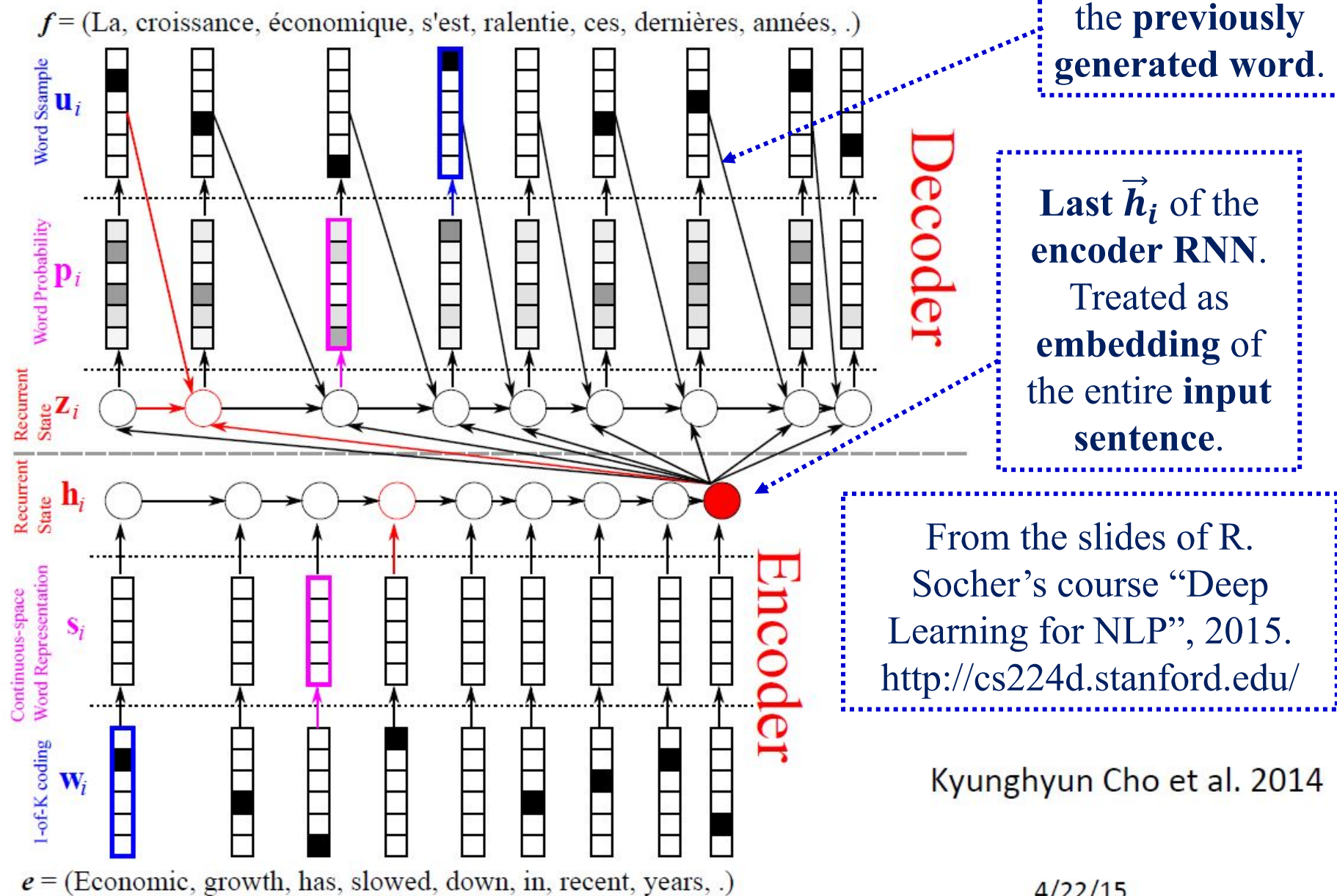
$$y_t = \text{softmax}(W^{(S)}h_t)$$

Minimize cross entropy error for all target words conditioned on source words

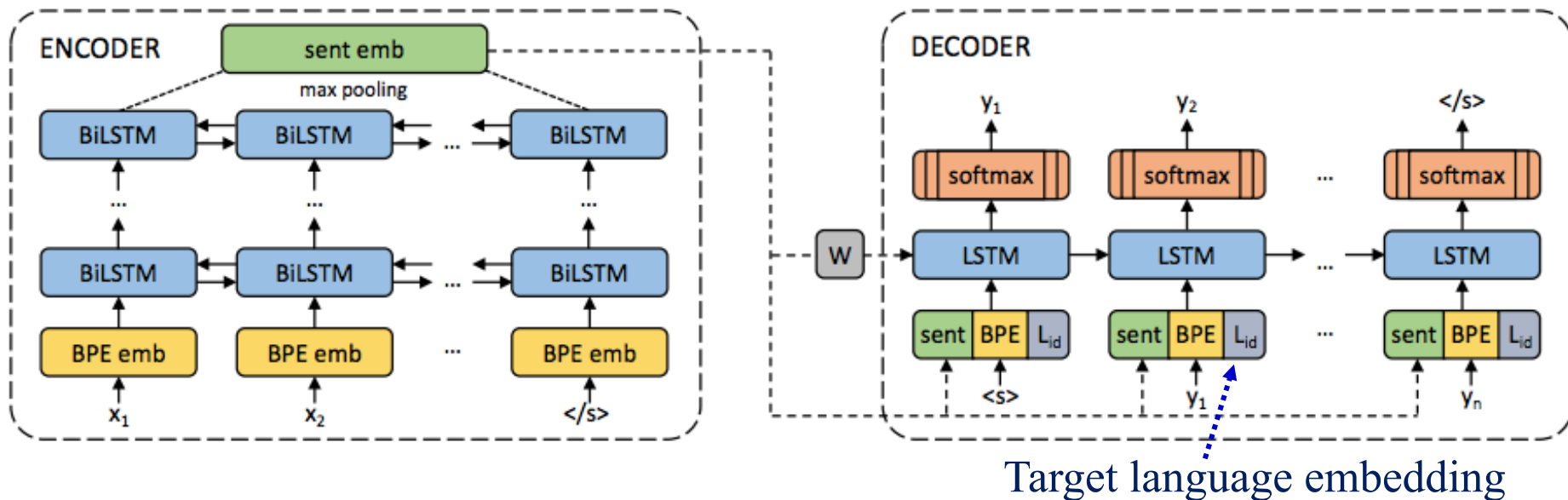
$$\max_{\theta} \frac{1}{N} \sum_{n=1}^N \log p_{\theta}(y^{(n)} | x^{(n)})$$



Different picture, same idea



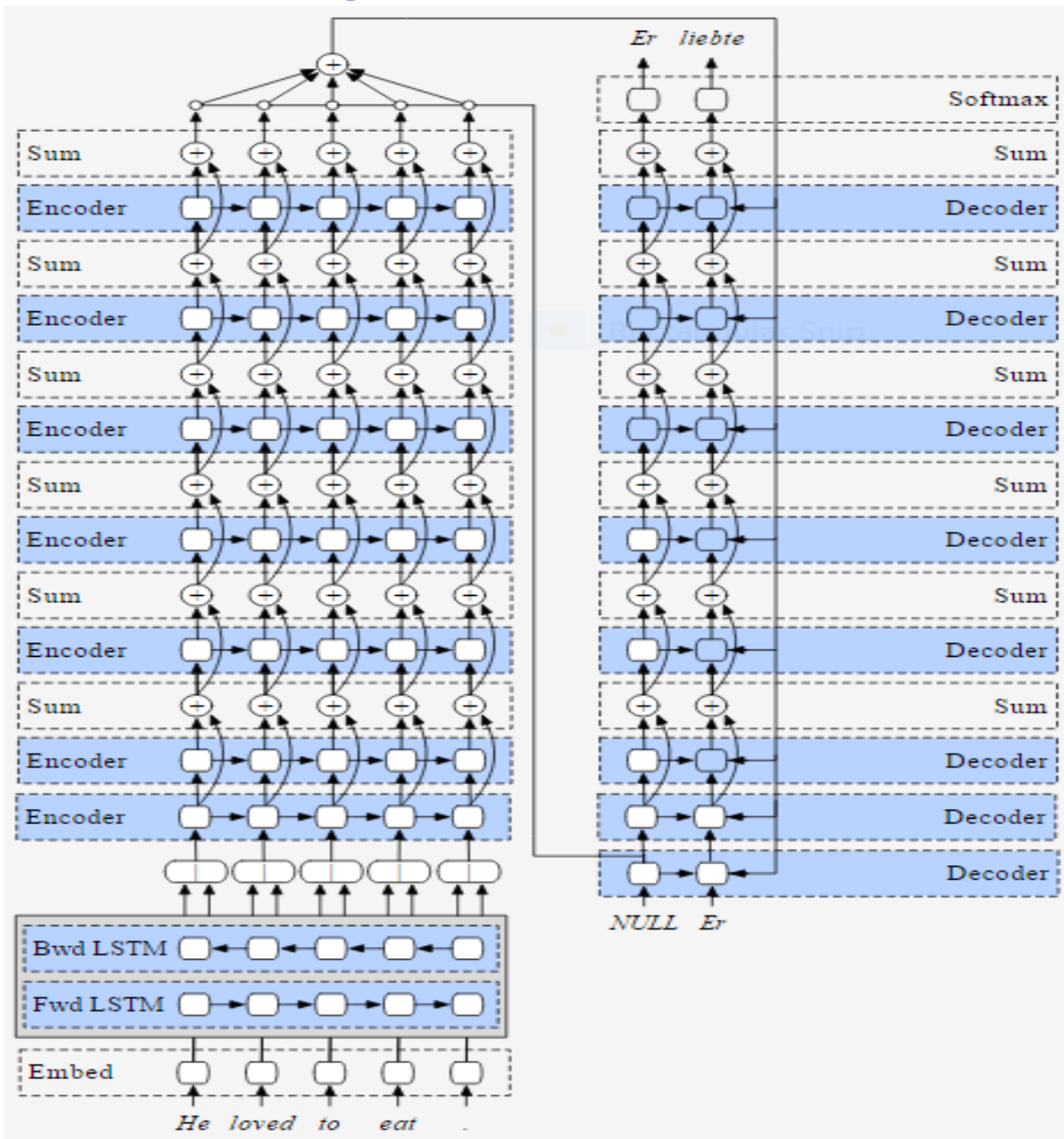
Universal sentence encoders



- **Laser:** Trained on parallel corpora of 93 languages.
 - Using the **same encoder** and **decoder** for all languages.
 - **Shared vocabulary** of sub-word units (BPEs).
- E.g., we can **train a classifier** on **English tweets**, and use the **same trained classifier** on **Greek tweets**.
 - In **both languages**, we use the **same encoder** to convert the each **tweet** to a **feature vector**.

M. Artetxe and H. Schwenk, “Massively Multilingual Sentence Embeddings for Zero-Shot Cross-Lingual Transfer and Beyond”. <https://arxiv.org/abs/1812.10464>
<https://code.fb.com/ai-research/laser-multilingual-sentence-embeddings/>

Google's RNN-based MT system

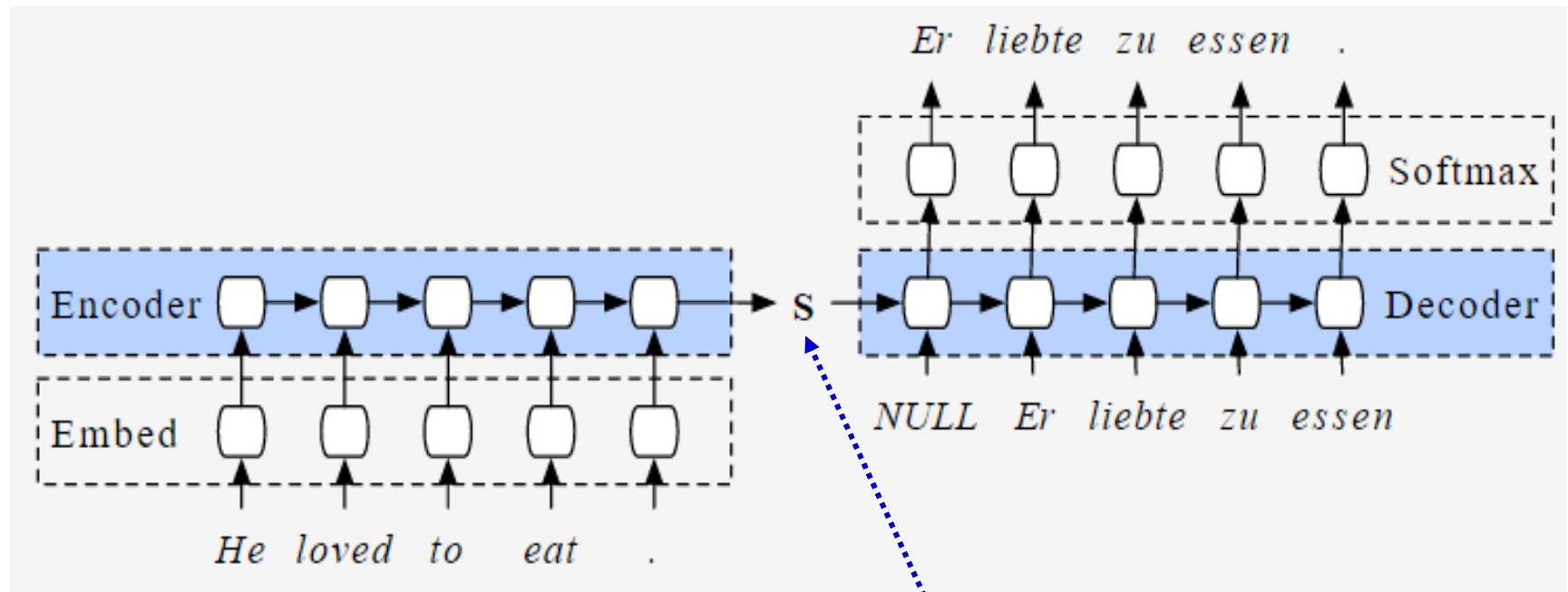


Google's paper:
<https://arxiv.org/abs/1609.08144>

Images from Stephen Merity's
http://smerity.com/articles/2016/google_nmt_arch.html

Easier to explain
step by step...

Basic Encoder-Decoder NMT



The **last state of the encoder** tries to represent the meaning of the **entire source sentence**.

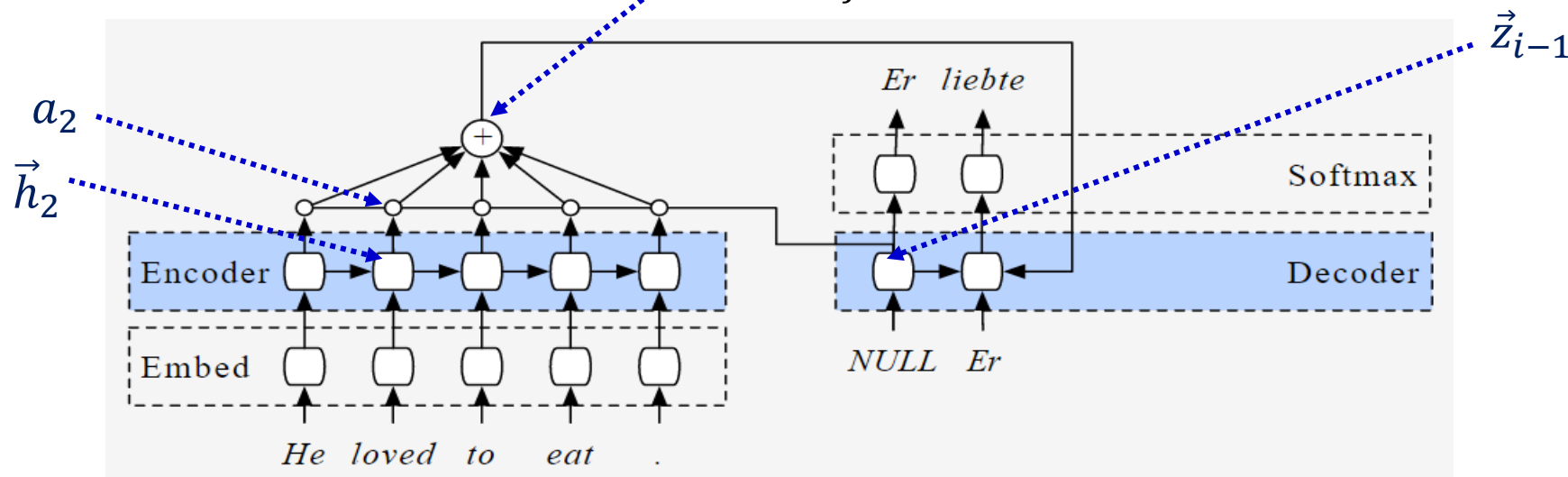
Google's paper: <https://arxiv.org/abs/1609.08144>

Images from Stephen Merity's http://smerity.com/articles/2016/google_nmt_arch.html

Encoder-Decoder with attention

The **source sentence** is now represented by the **weighted sum** of the **encoder states**:

$$\vec{h}_{sum} = \sum_j a_j \vec{h}_j$$



For each German word, the **attention scores** over the English words **change**! Each “**attention**” weight a_j is a **function** of the **corresponding encoder state** $\vec{h}_j$ and the **previous state** $\vec{z}_{i-1}$ of the **decoder** (memory of translation so far), e.g.:

$$\tilde{a}_j = f(W\vec{h}_j + U\vec{z}_{i-1}), \quad a_j = \text{softmax}(\tilde{a}_j)$$

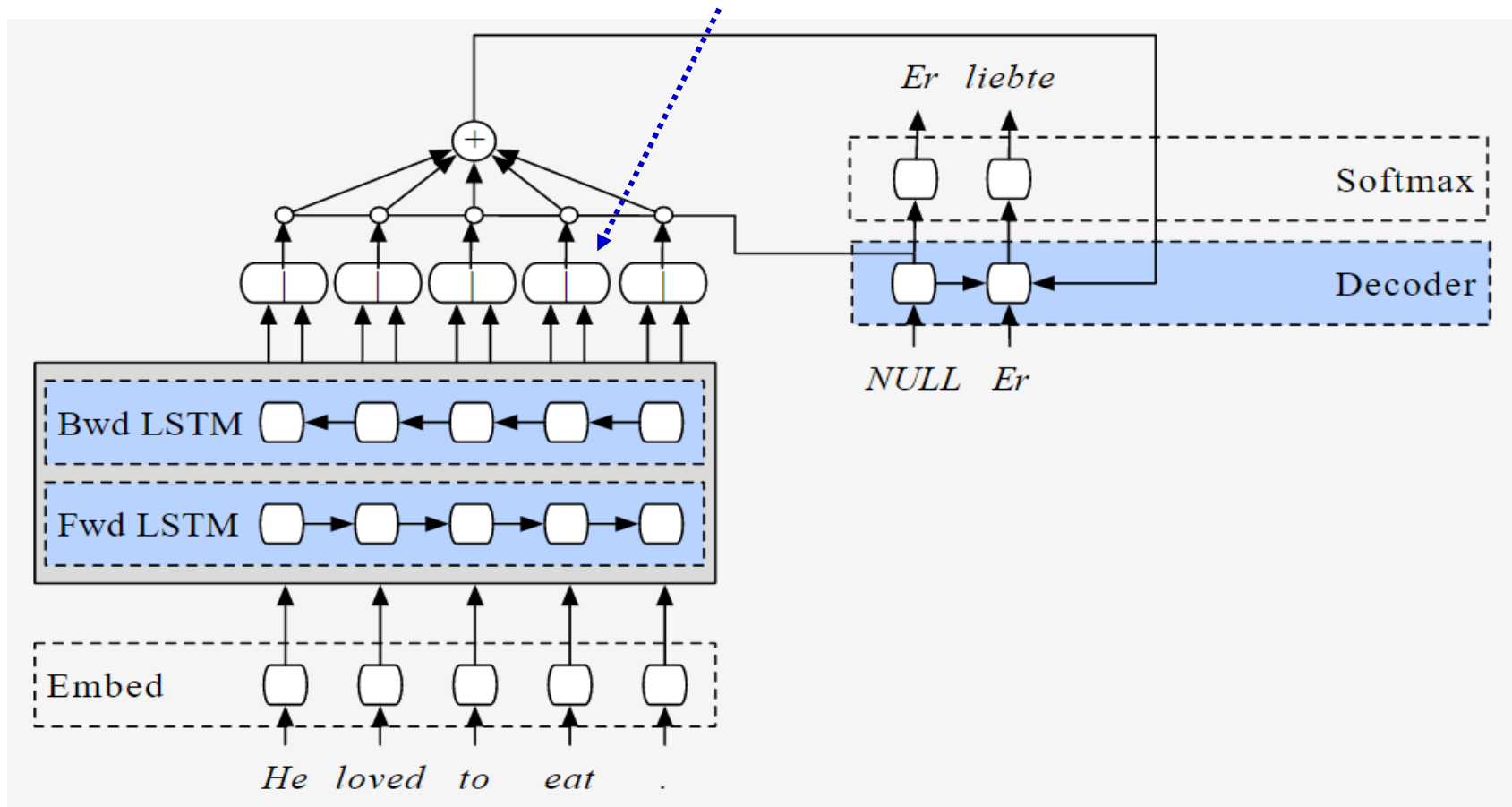
with a **softmax** to make the a_j weights sum to 1.

Google’s paper: <https://arxiv.org/abs/1609.08144>

Images from Stephen Merity’s http://smerity.com/articles/2016/google_nmt_arch.html

Bidirectional LSTM encoder

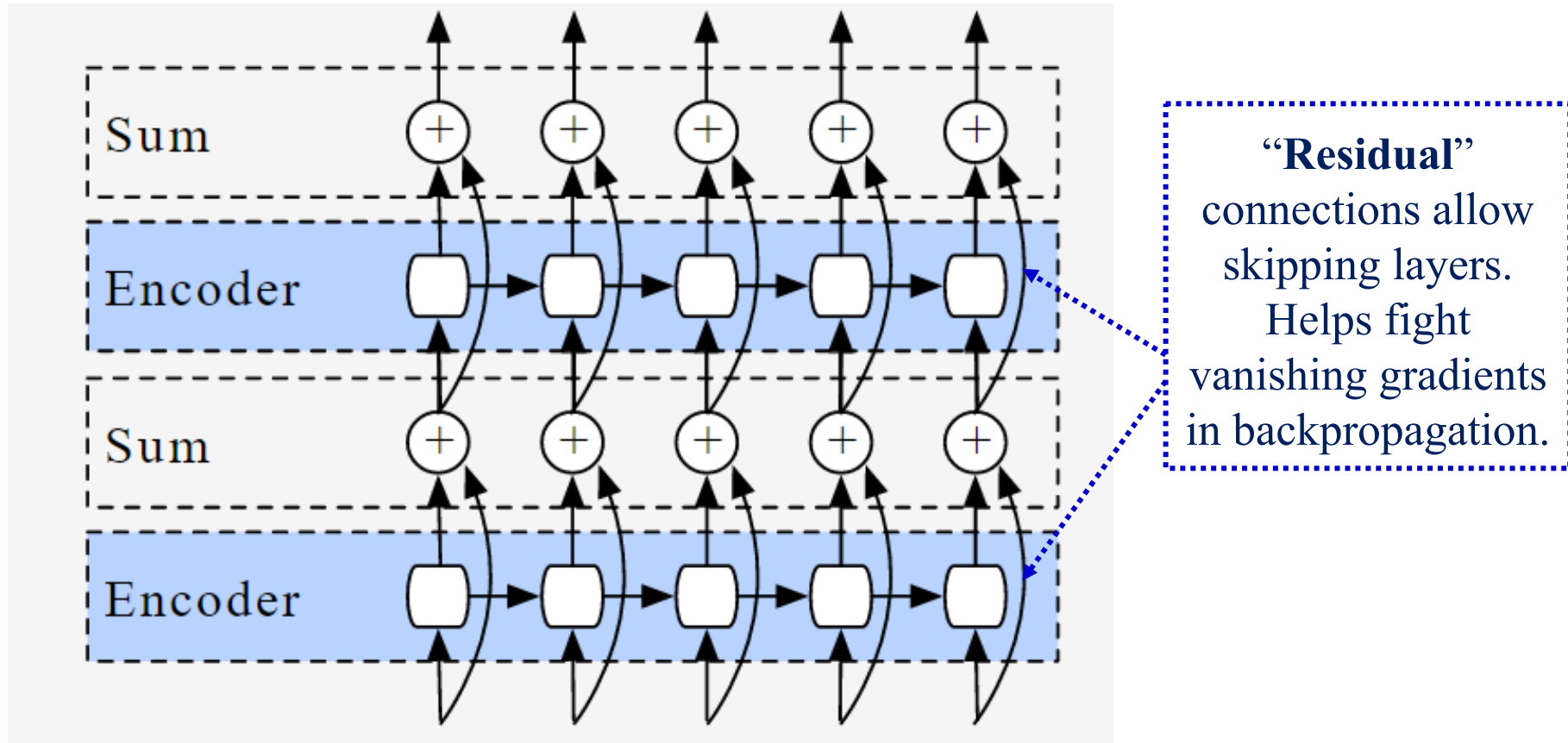
The encoder is now a **bidirectional LSTM**. The **encoder state** for the j -th word of **the source** sentence is the **concatenation** of the **corresponding states** of the **forward and backward LSTM**.



Google's paper: <https://arxiv.org/abs/1609.08144>

Images from Stephen Merity's http://smerity.com/articles/2016/google_nmt_arch.html

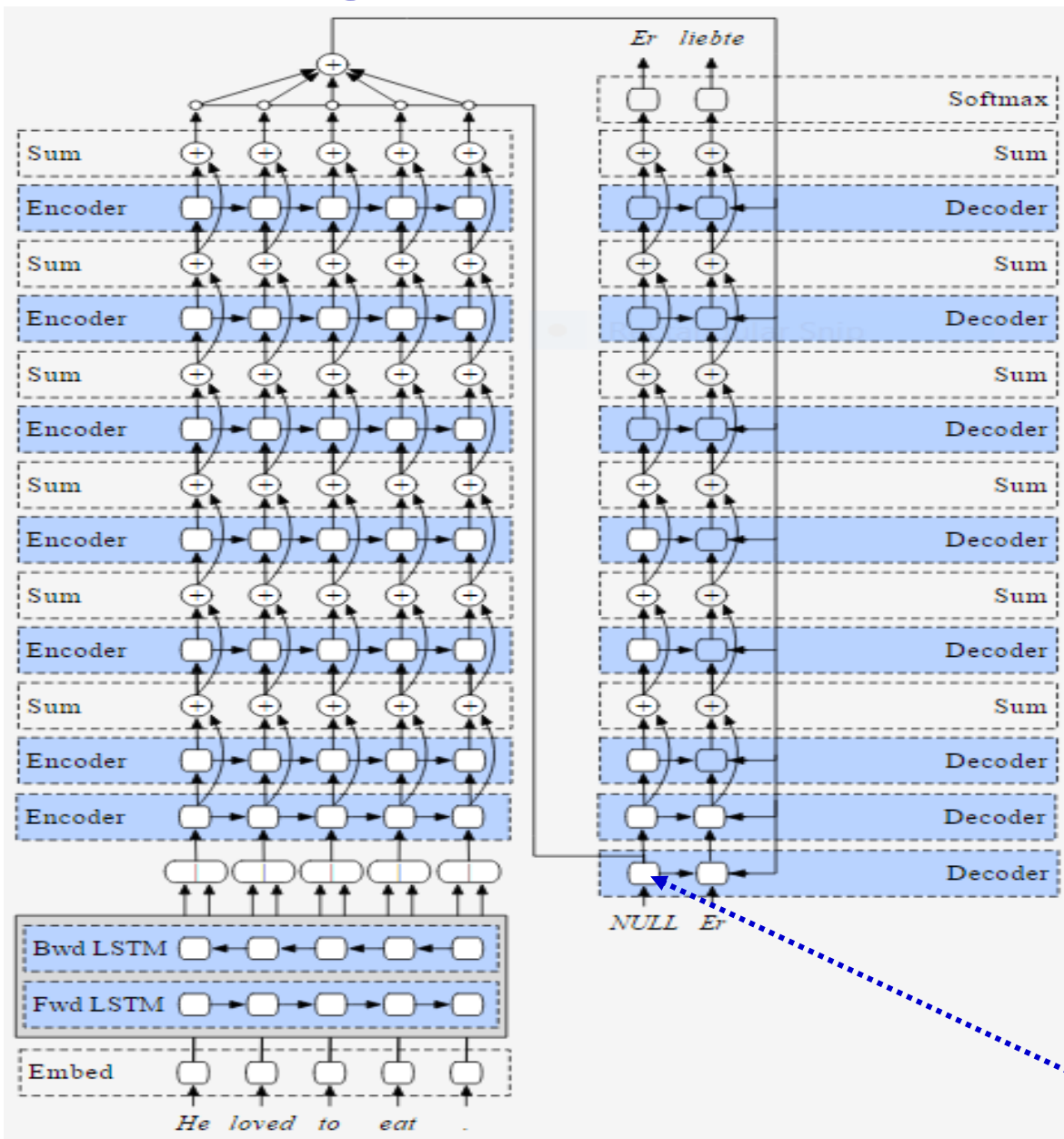
Stacking RNNs and residuals



Google’s paper: <https://arxiv.org/abs/1609.08144>

Images from Stephen Merity’s http://smerity.com/articles/2016/google_nmt_arch.html

Google's RNN-based MT system

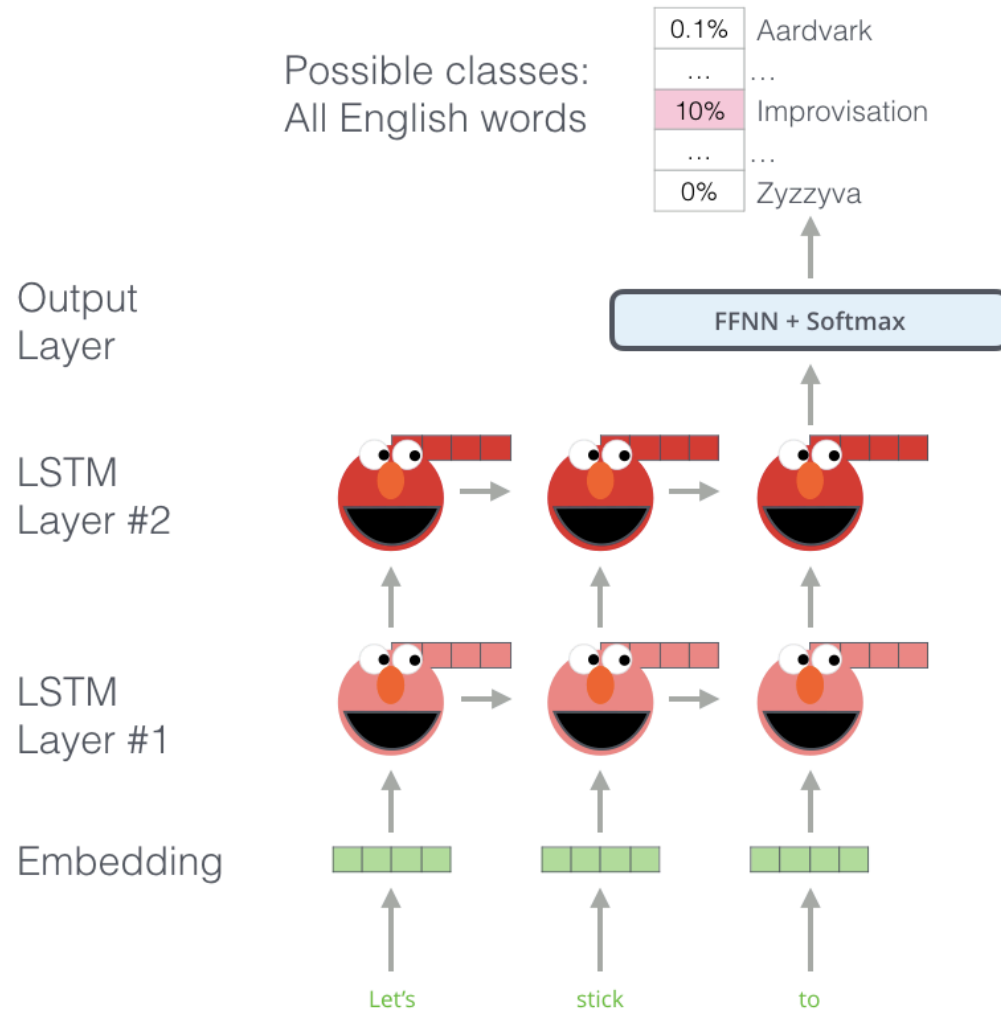


Google's paper:
<https://arxiv.org/abs/1609.08144>

Images from Stephen Merity's
http://smerity.com/articles/2016/google_nmt_arch.html

Attention based on the previous state of the bottom decoder only, to speed up computations.

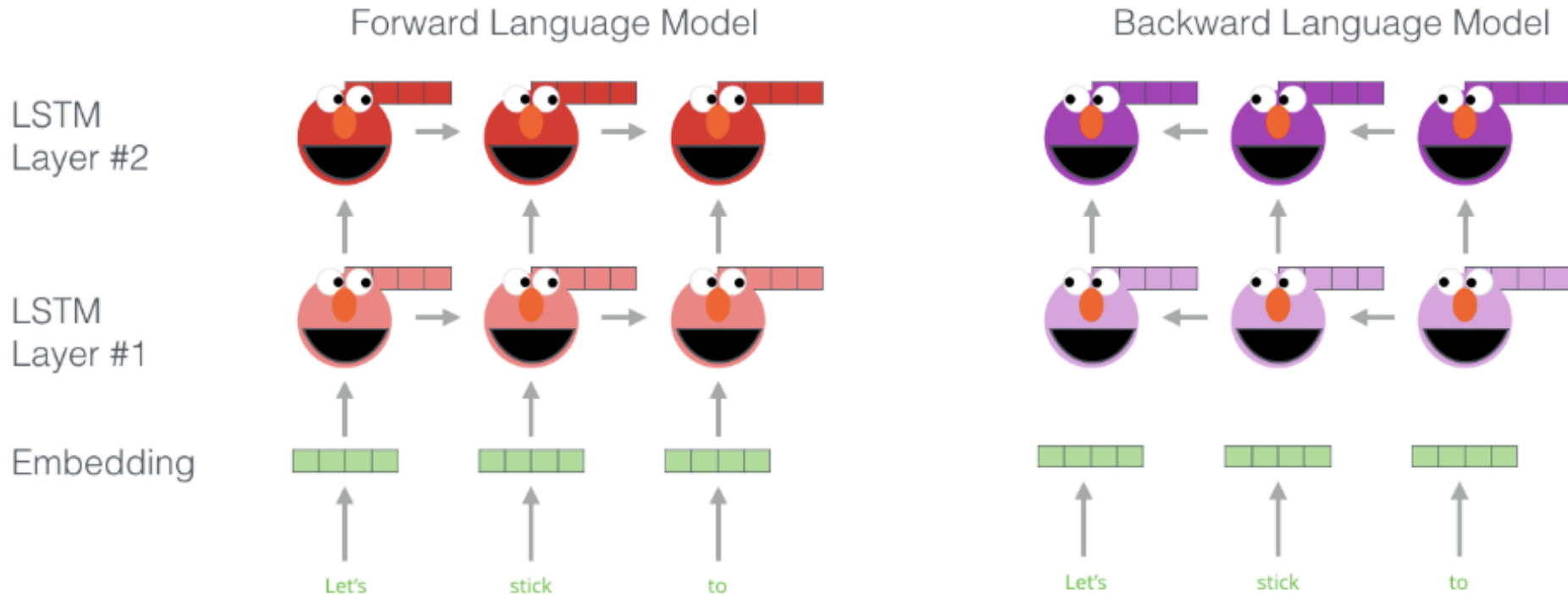
ELMo – Pretraining LMs to obtain context aware word embeddings



Figures from J. Alammr's "The Illustrated BERT, ELMo, and co."

<http://jalammar.github.io/illustrated-bert/>. ELMo paper: Peters et al. "Deep Contextualized Word Representations", NAACL-HLT 2018. <http://aclweb.org/anthology/N18-1202>

ELMo – Pretraining LMs to obtain context aware word embeddings



Figures from J. Alammr's "The Illustrated BERT, ELMo, and co."

<http://jalammar.github.io/illustrated-bert/>. ELMo paper: Peters et al. "Deep Contextualized Word Representations", NAACL-HLT 2018. <http://aclweb.org/anthology/N18-1202>

ELMo – Pretraining LMs to obtain context aware word embeddings

Embedding of “stick” in “Let’s stick to” - Step #2

1- Concatenate hidden layers



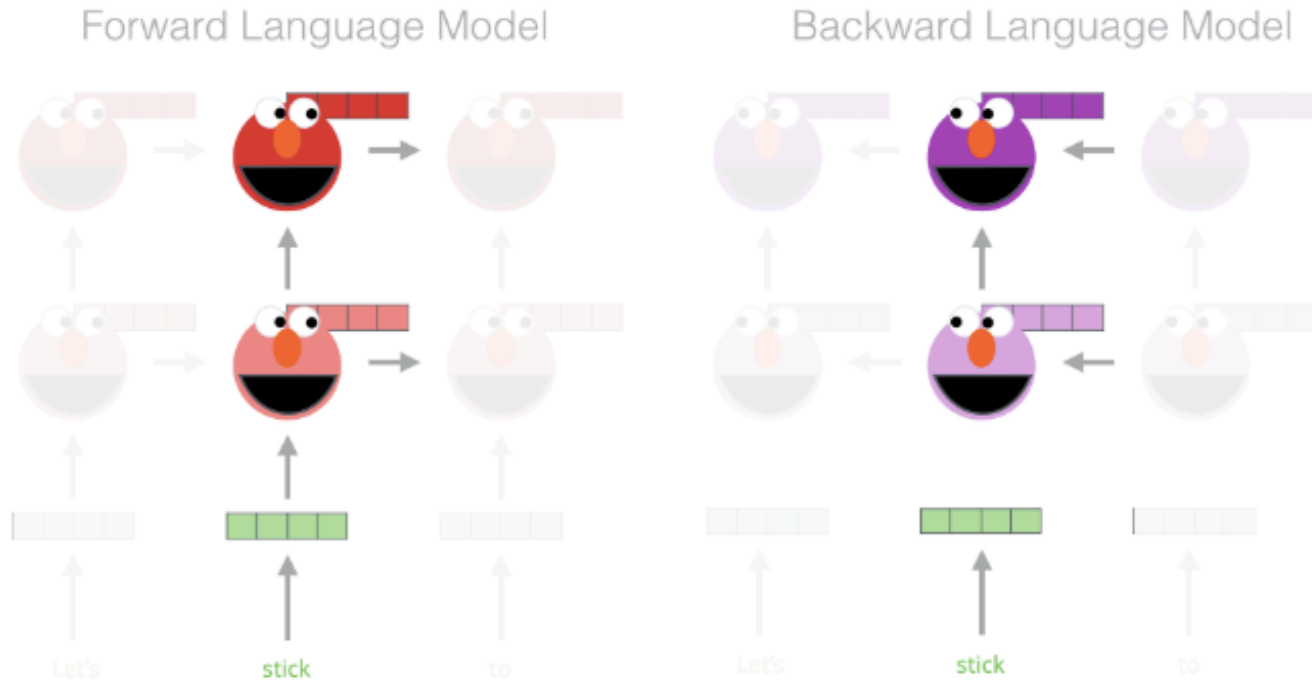
2- Multiply each vector by a weight based on the task



3- Sum the (now weighted) vectors



ELMo embedding of “stick” for this task in this context

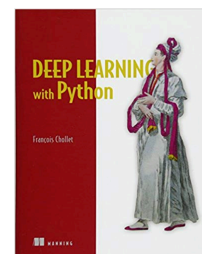
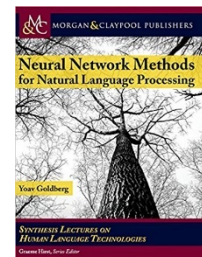


Figures from J. Alammr's “The Illustrated BERT, ELMo, and co.”

<http://jalammar.github.io/illustrated-bert/>. ELMo paper: Peters et al. “Deep Contextualized Word Representations”, NAACL-HLT 2018. <http://aclweb.org/anthology/N18-1202>

Recommended reading

- Y. Goldberg, *Neural Network Models for Natural Language Processing*, Morgan & Claypool Publishers, 2017.
 - Mostly chapters 14–17 .
- Jurafsky and Martin's, *Speech and Language Processing* is being revised (3rd edition) to include Deep Learning methods.
 - <http://web.stanford.edu/~jurafsky/slp3/>
- For an introduction to Keras (see labs) and practical DL for NLP and vision, see F. Chollet's *Deep Learning in Python*, Manning Publications, 2017.
- For an introduction to PyTorch and hands-on tutorials for NLP and vision, see the fast.ai free courses.
 - <https://course.fast.ai/>



Recommended reading

- Stanford's *NLP with Deep Learning* course.
 - Course material: <http://web.stanford.edu/class/cs224n/>
 - Videos: available on YouTube.
- CMU's *Neural Networks for NLP* course.
 - See: <http://www.phontron.com/class/nn4nlp2019/>
 - Videos: available on YouTube.